



A Gentle Introduction to Ownership Types in Rust

Papers We LoveSM $f(x)=x$

Daniel Galán Pascual 

```
fn main() {  
    let mut v = vec![1, 2, 3];  
  
    {  
        let first = &v[0];  
  
        println!("{}", first);  
  
        v.push(4);  
        println!("{:?}", v);  
    }  
}
```

« — v ↑ +R +W +O

« — v → R W Ø
first ↑ +R - +O
*first ↑ +R - -

« — v ↻ R +W +O
first ↓ R - Ø
*first ↓ R - -

« — v ↓ R



About me

- Doctoral student at **ETH***zürich*
 - Information Security Group
- Interests:
 - Safety and security by construction
 - Programming language theory
 - Type systems
 - Static analysis
 - Information flow control

 **Haskell**

 **Scala**

 **WA**



About me

- Doctoral student at **ETH** zürich
 - Information Security Group
- Interests:
 - Safety and security by construction
 - Programming language theory
 - Type systems
 - Static analysis
 - Information flow control

 **Haskell**

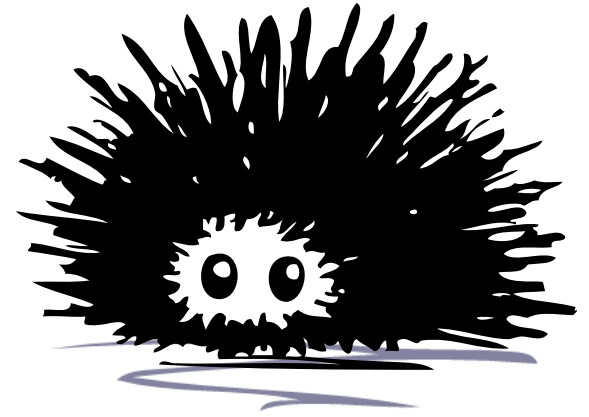
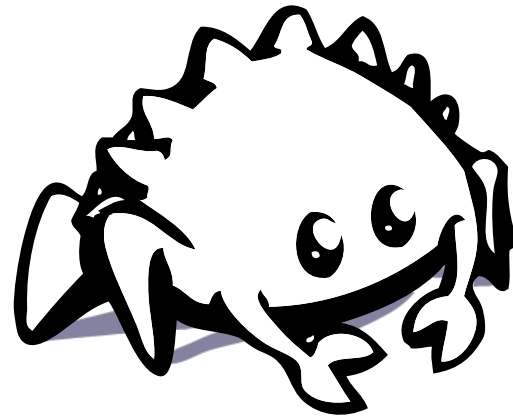
 **Scala**

 **WA**



I spend my days making compilers reject programs. Today I will explain why that is a good thing.

Why Rust?



The problem

The problem

^[1] Microsoft Security Response Center. "A proactive approach to more secure code." MSRC Blog, 2019.

<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

^[2] CISA. "The Urgent Need for Memory Safety in Software Products." cisa.gov, 2023.

<https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>

^[3] Chromium Project. "Memory Safety." chromium.org. <https://www.chromium.org/Home/chromium-security/memory-safety/>

- **~70%** of serious security vulnerabilities at Microsoft, Google, and Mozilla are memory safety bugs^[1-3]

The problem

[1] Microsoft Security Response Center. "A proactive approach to more secure code." MSRC Blog, 2019.

<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

[2] CISA. "The Urgent Need for Memory Safety in Software Products." cisa.gov, 2023.

<https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>

[3] Chromium Project. "Memory Safety." chromium.org. <https://www.chromium.org/Home/chromium-security/memory-safety/>

[4] Prossimo / ISRG. "What is memory safety and why does it matter?" memorysafety.org.

<https://www.memorysafety.org/docs/memory-safety/>

- **~70%** of serious security vulnerabilities at Microsoft, Google, and Mozilla are memory safety bugs^[1-3]
- **More than 80% of zero-days** exploited in the wild are memory safety issues^[4]

The problem



CURL AND LIBCURL

HOW I MADE A HEAP OVERFLOW IN CURL

🕒 OCTOBER 11, 2023 👤 DANIEL STENBERG 💬 25 COMMENTS

In association with the release of [curl 8.4.0](#), we publish a security advisory and all the details for [CVE-2023-38545](#). This problem is the worst security problem found in curl in a long time. We set it to severity **HIGH**.

[1] Microsoft Security Response Center. "A proactive approach to more secure code." MSRC Blog, 2019.

<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

[2] CISA. "The Urgent Need for Memory Safety in Software Products." cisa.gov, 2023.

<https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>

[3] Chromium Project. "Memory Safety." chromium.org. <https://www.chromium.org/Home/chromium-security/memory-safety/>

[4] Prossimo / ISRG. "What is memory safety and why does it matter?" memorysafety.org.

<https://www.memorysafety.org/docs/memory-safety/>

[5] Stenberg, D. "How I made a heap overflow in curl." <https://daniel.haxx.se/blog/2023/10/11/how-i-made-a-heap-overflow-in-curl/>

- ~**70%** of serious security vulnerabilities at Microsoft, Google, and Mozilla are memory safety bugs^[1-3]
- **More than 80% of zero-days** exploited in the wild are memory safety issues^[4]
- Daniel Stenberg, creator of **curl**, introduced a heap overflow undetected for 1315 days^[5]

The problem



CURL AND LIBCURL

HOW I MADE A HEAP OVERFLOW IN CURL

🕒 OCTOBER 11, 2023 👤 DANIEL STENBERG 💬 25 COMMENTS

In association with the release of [curl 8.4.0](#), we publish a security advisory and all the details for [CVE-2023-38545](#). This problem is the worst security problem found in curl in a long time. We set it to severity **HIGH**.

[1] Microsoft Security Response Center. "A proactive approach to more secure code." MSRC Blog, 2019.

<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

[2] CISA. "The Urgent Need for Memory Safety in Software Products." cisa.gov, 2023.

<https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>

[3] Chromium Project. "Memory Safety." chromium.org. <https://www.chromium.org/Home/chromium-security/memory-safety/>

[4] Prossimo / ISRG. "What is memory safety and why does it matter?" memorysafety.org.

<https://www.memorysafety.org/docs/memory-safety/>

[5] Stenberg, D. "How I made a heap overflow in curl." <https://daniel.haxx.se/blog/2023/10/11/how-i-made-a-heap-overflow-in-curl/>

- ~**70%** of serious security vulnerabilities at Microsoft, Google, and Mozilla are memory safety bugs^[1-3]
- **More than 80% of zero-days** exploited in the wild are memory safety issues^[4]
- Daniel Stenberg, creator of **curl**, introduced a heap overflow undetected for 1315 days^[5]
- **Systems programming languages such as C or C++ offer little or no guardrails**

The problem



CURL AND LIBCURL

HOW I MADE A HEAP OVERFLOW IN CURL

🕒 OCTOBER 11, 2023 👤 DANIEL STENBERG 💬 25 COMMENTS

In association with the release of [curl 8.4.0](#), we publish a security advisory and all the details for [CVE-2023-38545](#). This problem is the worst security problem found in curl in a long time. We set it to severity **HIGH**.

[1] Microsoft Security Response Center. "A proactive approach to more secure code." MSRC Blog, 2019.

<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

[2] CISA. "The Urgent Need for Memory Safety in Software Products." cisa.gov, 2023.

<https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>

[3] Chromium Project. "Memory Safety." chromium.org. <https://www.chromium.org/Home/chromium-security/memory-safety/>

[4] Prossimo / ISRG. "What is memory safety and why does it matter?" memorysafety.org.

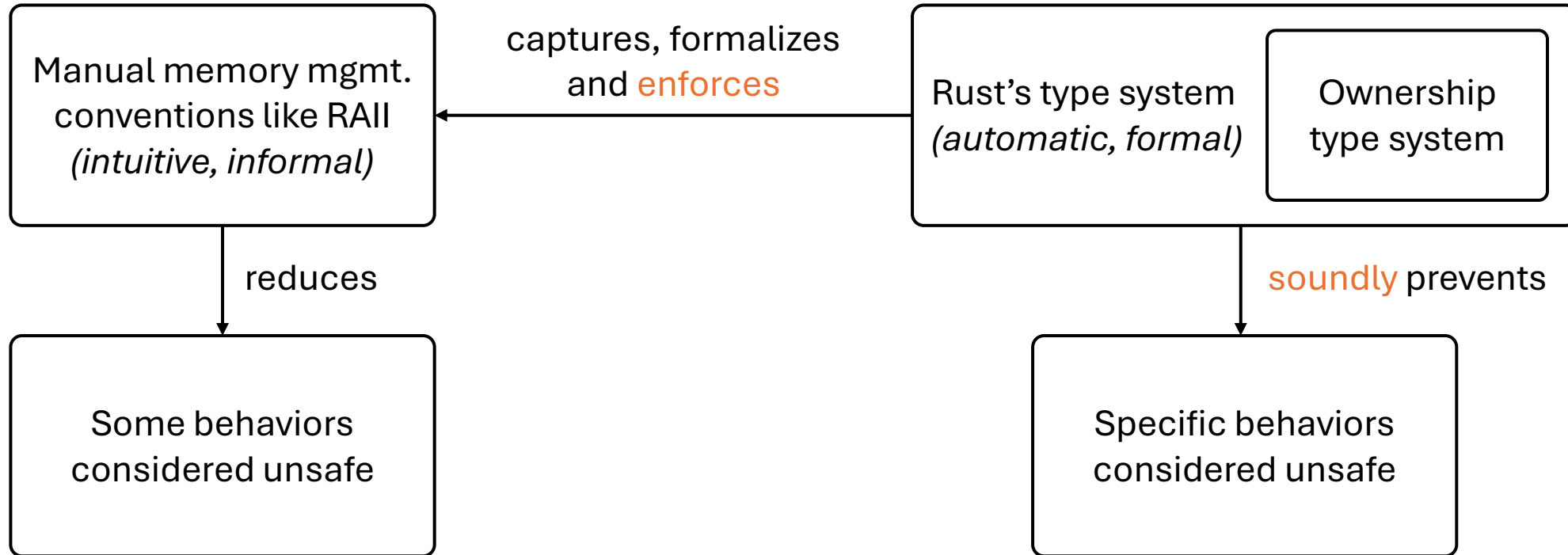
<https://www.memorysafety.org/docs/memory-safety/>

[5] Stenberg, D. "How I made a heap overflow in curl." <https://daniel.haxx.se/blog/2023/10/11/how-i-made-a-heap-overflow-in-curl/>

[6] Vander Stoep, J. and Rebert, A. "Eliminating Memory Safety Vulnerabilities at the Source." Google Security Blog, September 2024. <https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>

- ~**70%** of serious security vulnerabilities at Microsoft, Google, and Mozilla are memory safety bugs^[1-3]
- **More than 80% of zero-days** exploited in the wild are memory safety issues^[4]
- Daniel Stenberg, creator of **curl**, introduced a heap overflow undetected for 1315 days^[5]
- **Systems programming languages such as C or C++ offer little or no guardrails**
- Android's memory safety vulnerabilities dropped from **76%** (2019) to **24%** (2024) after adopting Rust^[6]

Ownership discipline in Rust



Ownership type theory



$\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi_e} p \Rightarrow \{ \overline{\omega p'} \}$ where $\Delta; \Gamma; \Theta \vdash_{\omega} p \Rightarrow \{ \overline{\omega p'} \}$ means $\Delta; \Gamma; \Theta \vdash_{\omega}^{\bullet} p \Rightarrow \{ \overline{\omega p'} \}$.

O-SAFEPLACE

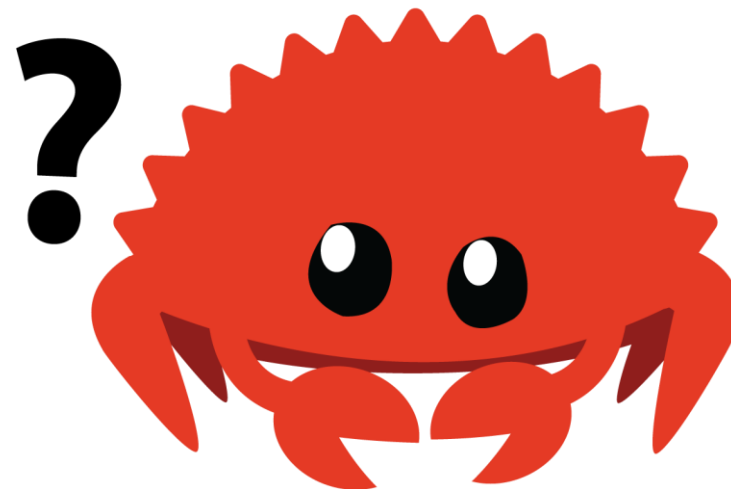
$$\frac{\begin{array}{l} \forall r' \mapsto \{ \bar{\ell} \} \in \text{regions}(\Gamma, \Theta). (\forall \omega' p^{\square}[\pi'] \in \{ \bar{\ell} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies \pi' \# \pi) \\ \vee (\exists \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma) \wedge \nexists \&r' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma). \pi' \in \{ \overline{\pi_e} \})) \end{array}}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi_e} \pi \Rightarrow \{ \overline{\omega \pi} \}}$$

O-DEREF

$$\frac{\begin{array}{l} \Gamma(\pi) = \&r \omega_{\pi} \tau_{\pi} \quad \Gamma(r) = \{ \overline{\omega' p}^n \} \quad \text{excl} = \{ \pi_j \text{ where } j \in \{ 1, \dots, n \} \mid p_j = p_j^{\square}[*\pi_j] \} \quad \omega \lesssim \omega_{\pi} \\ \forall i \in \{ 1 \dots n \}. \Delta; \Gamma; \Theta \vdash_{\omega}^{\pi_e, \text{excl}, \pi} p^{\square}[p_i] \Rightarrow \{ \overline{\omega p'_i} \} \\ \forall r' \mapsto \{ \bar{\ell} \} \in \text{regions}(\Gamma, \Theta). (\forall \omega' p'' \in \{ \bar{\ell} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies p'' \# p^{\square}[*\pi]) \\ \vee (\exists \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma) \wedge \nexists \&r' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma). \pi' \in \{ \overline{\pi_e}, \text{excl}, \pi \})) \end{array}}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi_e} p^{\square}[*\pi] \Rightarrow \{ \overline{\omega p'_1}, \dots, \overline{\omega p'_n}, \overline{\omega p^{\square}[*\pi]} \}}$$

O-DEREFABS

$$\frac{\begin{array}{l} \Gamma(\pi) = \&q \omega_{\pi} \tau_{\pi} \quad \Delta; \Gamma \vdash_{\omega} p^{\square}[*\pi] : \tau \quad \omega \lesssim \omega_{\pi} \\ \forall r' \mapsto \{ \bar{\ell} \} \in \text{regions}(\Gamma, \Theta). (\forall \omega' p \in \{ \bar{\ell} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies p \# p^{\square}[*\pi]) \\ \vee (\exists \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma) \wedge \nexists \&r' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&r' \omega' \tau' \in \text{explode}(\Gamma). \pi' \in \{ \overline{\pi_e}, \pi \})) \end{array}}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi_e} p^{\square}[*\pi] \Rightarrow \{ \overline{\omega p^{\square}[*\pi]} \}}$$



Weiss, A., Gierczak, O., Patterson, D., Matsakis, N. D., and Ahmed, A. "Oxide: The Essence of Rust." arXiv preprint arXiv:1903.00982, 2019 (revised 2021). <https://arxiv.org/abs/1903.00982>

This session's paper

265

A Grounded Conceptual Model for Ownership Types in Rust

WILL CRICHTON, Brown University, USA
GAVIN GRAY, ETH Zürich, Switzerland
SHRIRAM KRISHNAMURTHI, Brown University, USA

1 [cs.PL] 8 Sep 2023

Programmers learning Rust struggle to understand ownership types, Rust's core mechanism for ensuring memory safety without garbage collection. This paper describes our attempt to systematically design a pedagogy for ownership types. First, we studied Rust developers' misconceptions of ownership to create the Ownership Inventory, a new instrument for measuring a person's knowledge of ownership. We found that Rust learners could not connect Rust's static and dynamic semantics, such as determining why an ill-typed program would (or would not) exhibit undefined behavior. Second, we created a conceptual model of Rust's semantics that explains borrow checking in terms of flow-sensitive permissions on paths into memory. Third, we implemented a Rust compiler plugin that visualizes programs under the model. Fourth, we integrated the permissions model and visualizations into a broader pedagogy of ownership by writing a new ownership chapter for *The Rust Programming Language*, a popular Rust textbook. Fifth, we evaluated an initial deployment of our pedagogy against the original version, using reader responses to the Ownership Inventory as a point of comparison. Thus far, the new pedagogy has improved learner scores on the Ownership Inventory by an average of 9% ($N = 342$, $d = 0.56$).

CCS Concepts: • **Theory of computation** → **Type structures**; • **Human-centered computing** → *Visualization systems and tools*; • **Social and professional topics** → *Computing education*.

Keywords: Rust, ownership types, conceptual model, ownership inventory

Crichton, W., Gray, G., Krishnamurthi, S. (2023). A grounded conceptual model for ownership types in Rust. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2), 1224-1252.

Introduction to **Ownership**



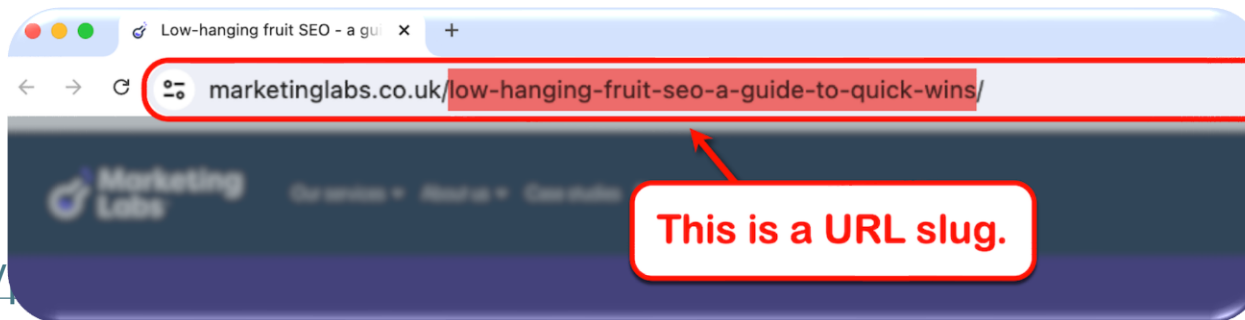
A mental model for memory

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", slug);  
11 }
```

<https://play.rust-lang.org/?gist=574097456d2bbc06eda7964c7a48de74>

A mental model for memory

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", slug);  
11 }
```

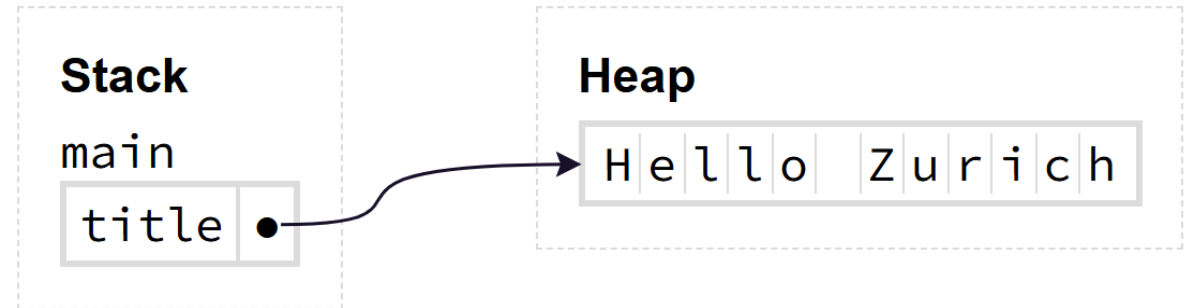


[https://](https://marketinglabs.co.uk/low-hanging-fruit-seo-a-guide-to-quick-wins/)

[e74](https://marketinglabs.co.uk/low-hanging-fruit-seo-a-guide-to-quick-wins/)

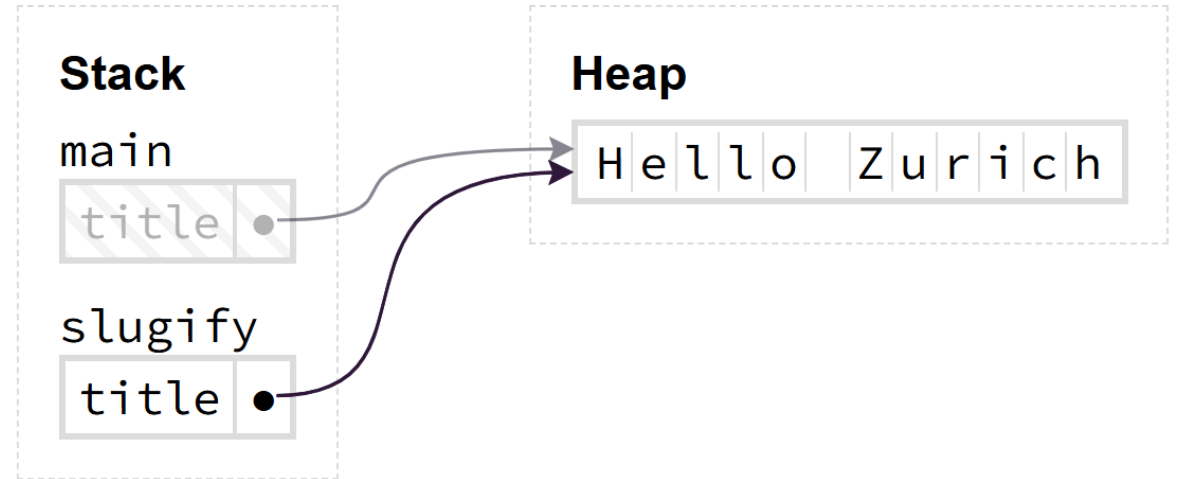
A mental model for memory

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", slug);  
11 }
```



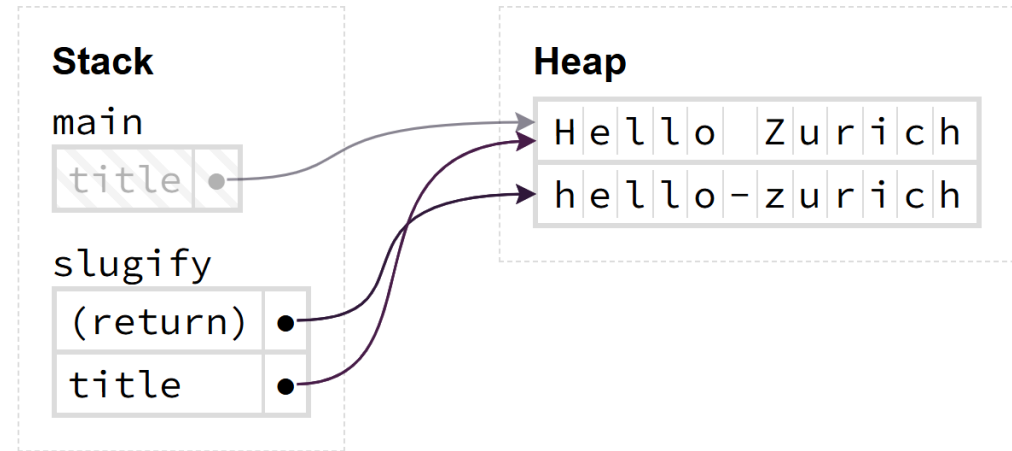
A mental model for memory

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", slug);  
11 }
```



A mental model for memory

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", slug);  
11 }
```



Implicit deallocation

```
1 {  
2   let a = String::from("Hello");  
3   let b = a;  
4   // implicit drop(b)  
5   // implicit drop(a)  
6 }
```

- Deallocation principle (almost correct) - - - - -

When a **binding** goes out of scope, its value is **dropped**.

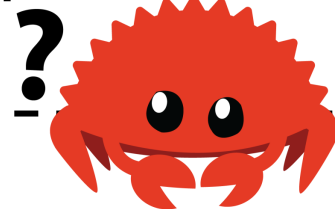
Implicit deallocation

```
1 {  
2   let a = String::from("Hello");  
3   let b = a;  
4   // implicit drop(b)  
5   // implicit drop(a)  
6 }
```

Aliasing causes double deallocations

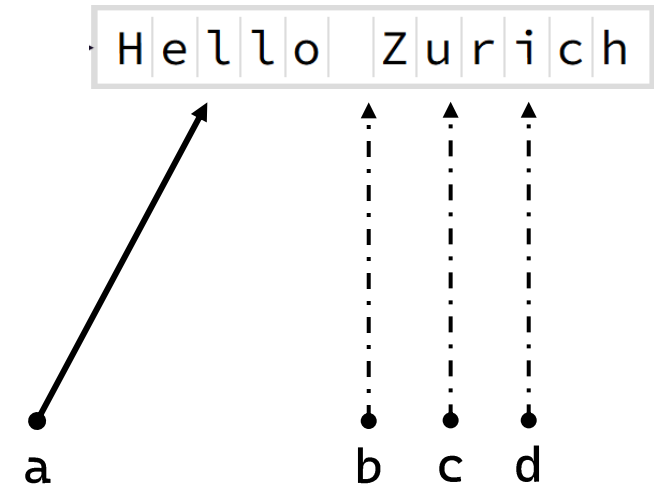
– Deallocation principle (almost correct) – – – – –

When a **binding** goes out of scope, its value is **dropped**.



Ownership

```
1 {  
2   let a = String::from("Hello");  
3   let b = &a;  
4   let c = &a;  
5   let d = &a;  
6 }
```

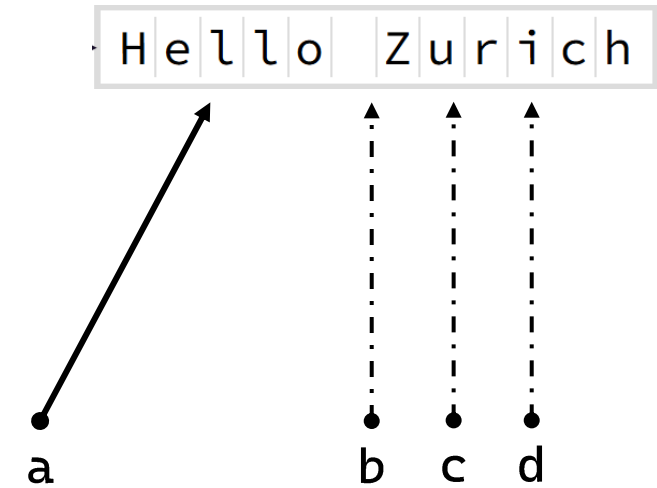


Owner
exactly one

(Immutable) Borrows
zero to many

Ownership

```
1 {  
2   let a = String::from("Hello");  
3   let b = &a;  
4   let c = &a;  
5   let d = &a;  
6 }
```



Owner
exactly one

(Immutable) Borrows
zero to many

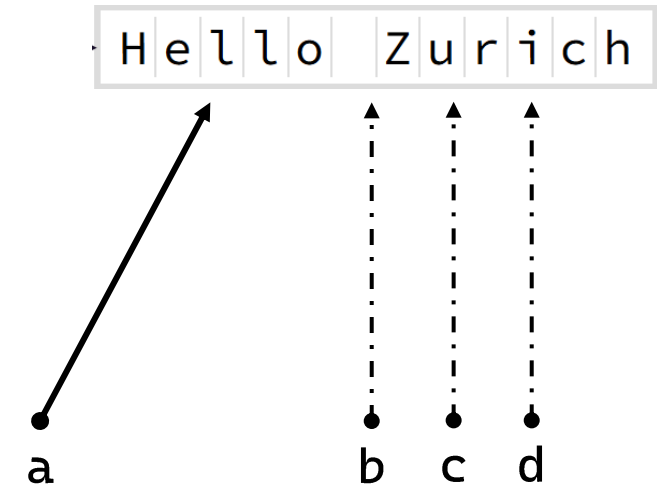
Ownership rule

Every **value** has:

- exactly **one owning binding**
- zero or more **non-owning (immutable) bindings**

Ownership

```
1 {  
2   let a = String::from("Hello");  
3   let b = &a;  
4   let c = &a;  
5   let d = &a;  
6 }
```



Owner
exactly one

(Immutable) Borrows
zero to many

(like *unique_ptr*)

(like *const T&* in C++11)

Ownership rule

Deallocation principle (fully correct)

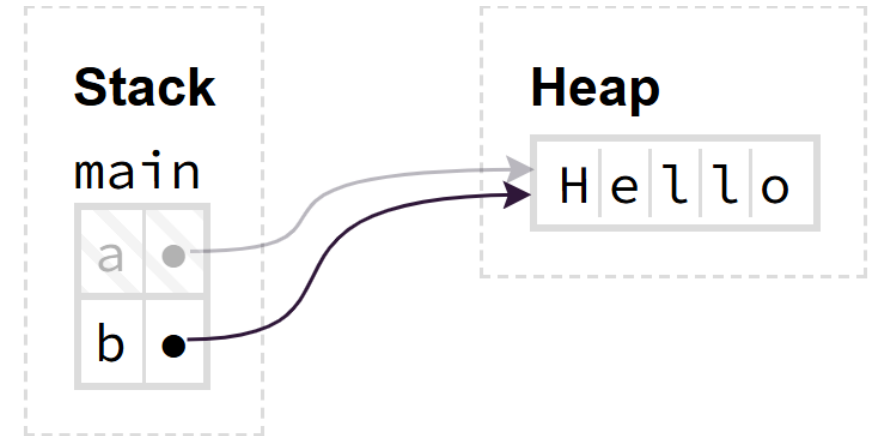
When an **owning binding** goes out of scope, its value is **dropped**.

Move semantics

```
1 let a = String::from("hello");  
2 let b = a;  
3 println!("{}", a);
```

Move semantics

```
1 let a = String::from("hello");  
2 let b = a;  
3 println!("{}", a);
```



Move semantics

On assignment:

- Transfer ownership into LHS
- Invalidate bindings in RHS

enforces

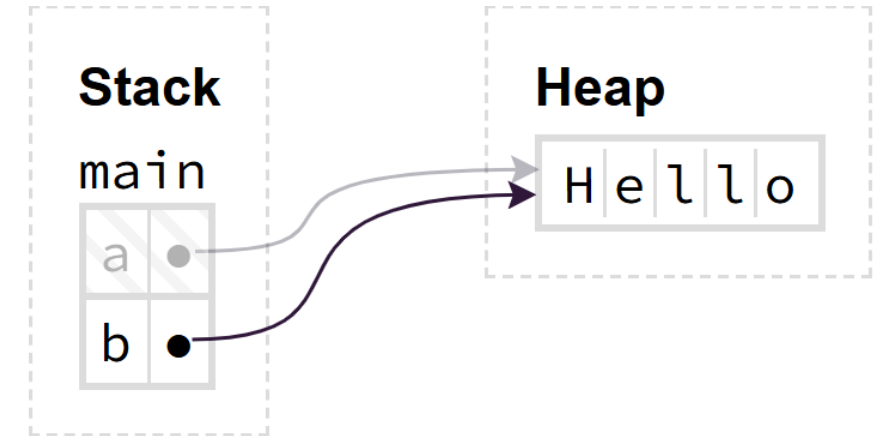
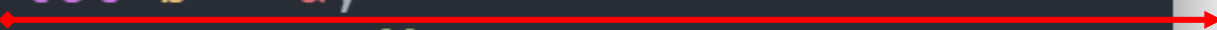
Ownership rule

Every **value** has exactly
one owning binding.

(like `std::move` in C++11)

Move semantics

```
1 let a = String::from("hello");  
2 let b = a;  
3 println!("{}", a);
```



Move semantics

On assignment:

- Transfer ownership into LHS
- Invalidate bindings in RHS

enforces

Ownership rule

Every **value** has exactly
one owning binding.

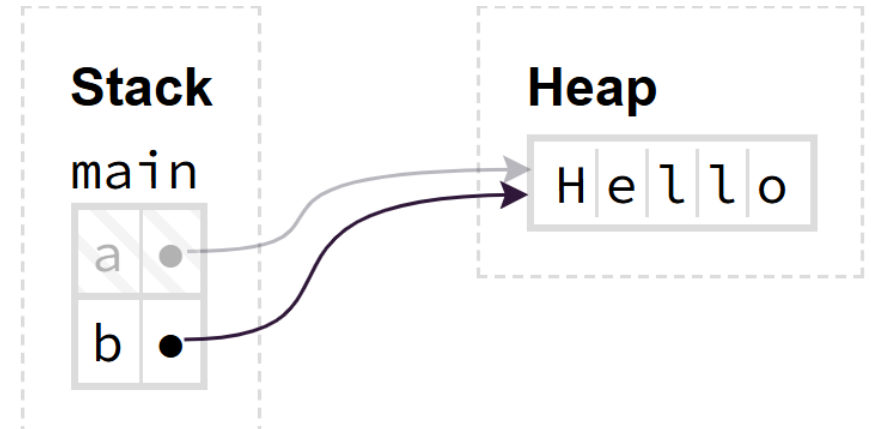
(like `std::move` in C++11)

Moved data principle

A variable **cannot be used** after it has been **moved** (invalidated).

Move semantics

```
1 let a = String::from("hello");
2 let b = a;
3 println!("{}", a);
```



```
error[E0382]: borrow of moved value: `a`
```

```
--> src/main.rs:4:18
```

```
2 | let a = String::from("hello");
  |       - move occurs because `a` has type `String`, which does not implement the `Copy` trait
3 | let b = a;
  |       - value moved here
4 | println!("{}", a);
  |               ^ value borrowed here after move
```

- Moved data principle

A variable **cannot be used** after it has been **moved** (invalidated).

Moving can be impractical

```
1 fn slugify(title: String) → String {
2     title
3     .to_lowercase()
4     .replace(' ', "-")
5 }
6
7 fn main() {
8     let title = input();
9     let slug = slugify(title);
10    println!("{}", slug);
11 }
```

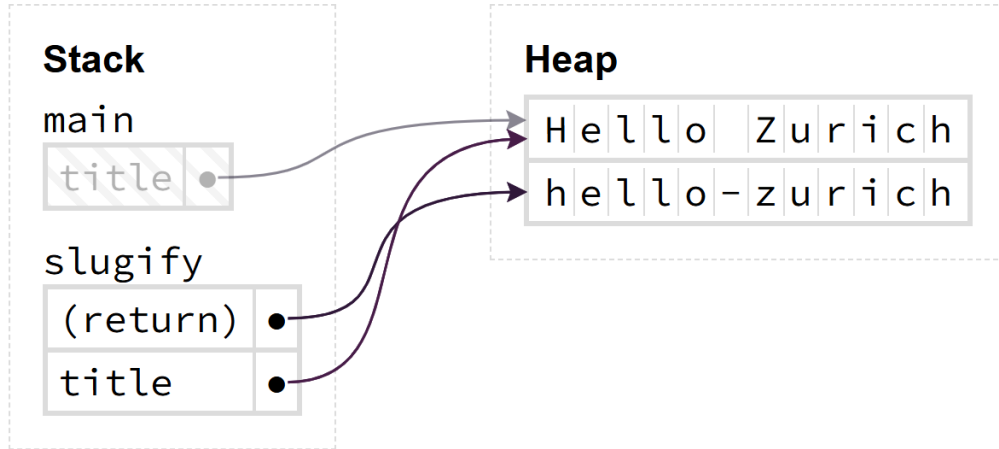
```
1 fn slugify(title: String) → String {
2     title
3     .to_lowercase()
4     .replace(' ', "-")
5 }
6
7 fn main() {
8     let title = input();
9     let slug = slugify(title);
10    println!("{}", title, slug);
11 }
```

error[E0382]: borrow of moved value: `title`

--> src/main.rs:18:25

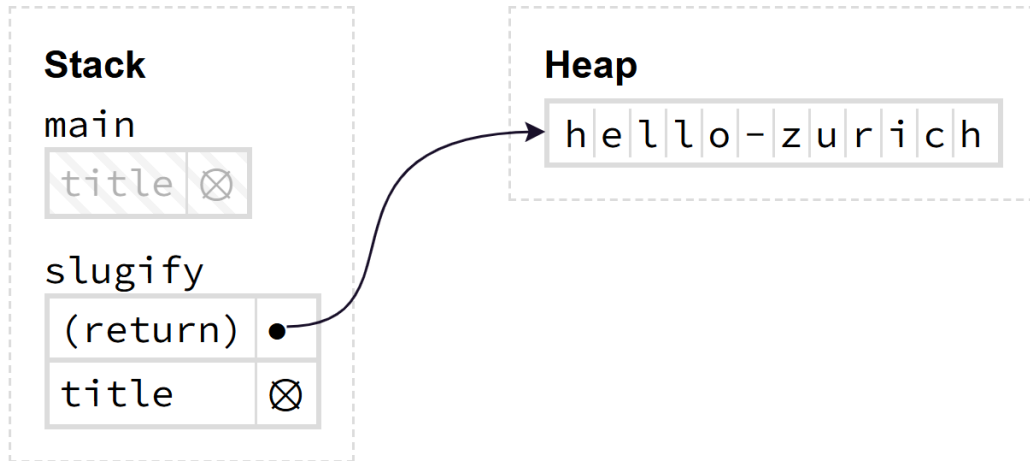
```
16 |     let title = input();
    |         ----- move occurs because `title` has type `String`, which does not implement the `Copy` trait
17 |     let slug = slugify(title);
    |                       ----- value moved here
18 |     println!("{}", title, slug);
    |                  ^^^^^ value borrowed here after move
```

Moving can be surprising



```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", title, slug);  
11 }
```

Moving can be surprising

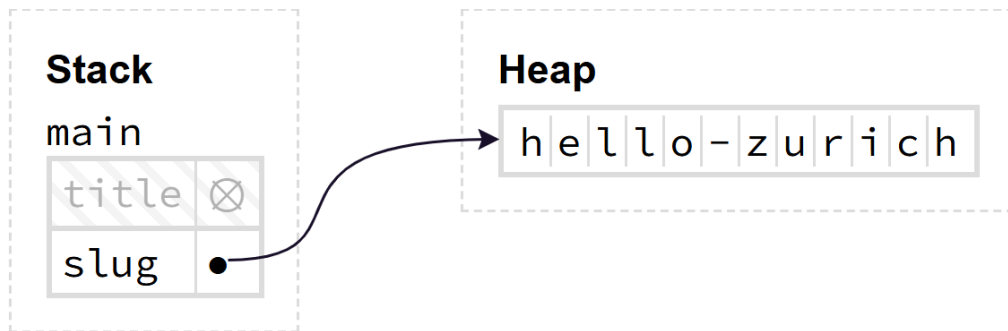


title deallocated alongside slugify's frame!

```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", title, slug);  
11 }
```

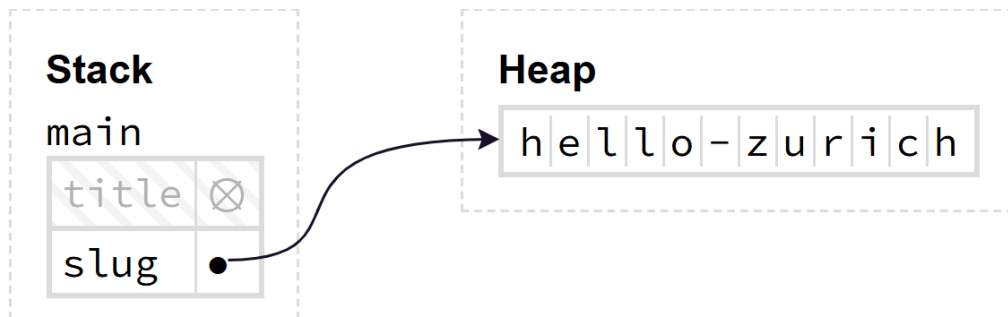
(leaving slugify)

Moving can be surprising



```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", title, slug);  
11 }
```

Moving can be surprising



```
1 fn slugify(title: String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(title);  
10    println!("{}", title, slug);  
11 }
```

error[E0382]: borrow of moved value: `title`

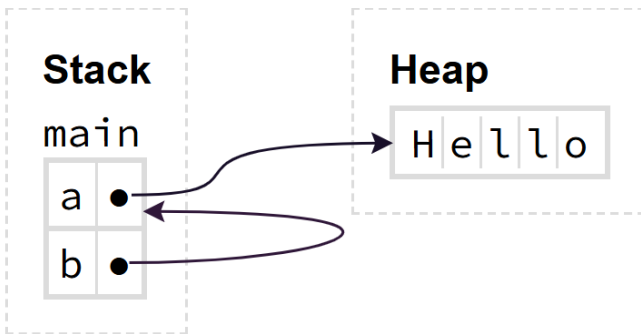
--> src/main.rs:18:25

```
16 |     let title = input();  
    |         ----- move occurs because `title` has type `String`, which does not implement the `Copy` trait  
17 |     let slug = slugify(title);  
    |                       ----- value moved here  
18 |     println!("{}", title, slug);  
    |                   ^^^^^ value borrowed here after move
```

Assignments are not so easy

Borrow / Share

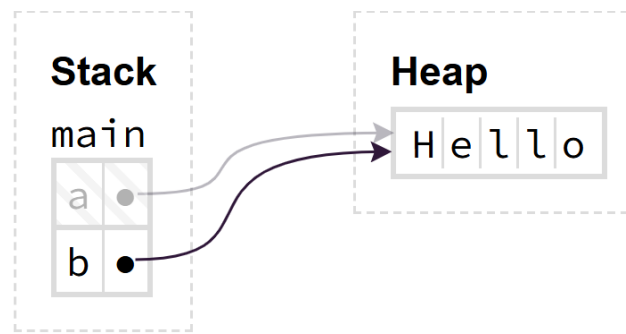
```
1 {  
2   let a = String::from("Hello");  
3   let b: &String = &a;  
4 }
```



- Zero cost
- Cedes ownership
- Possible lifetime issues

Move

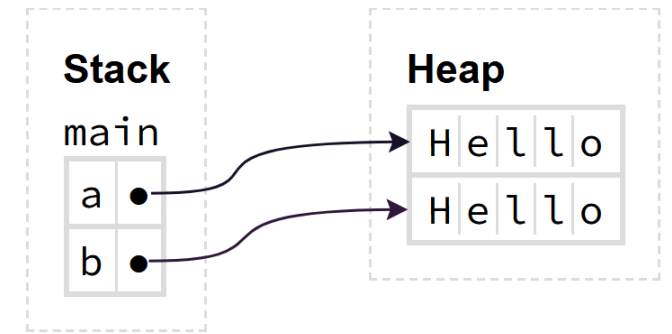
```
1 {  
2   let a = String::from("Hello");  
3   let b: String = a;  
4 }
```



- Zero cost
- Takes ownership
- Possibly impractical

Copy

```
1 {  
2   let a = String::from("Hello");  
3   let b: String = a.clone();  
4 }
```



- Cost scales with size
- Create new ownership

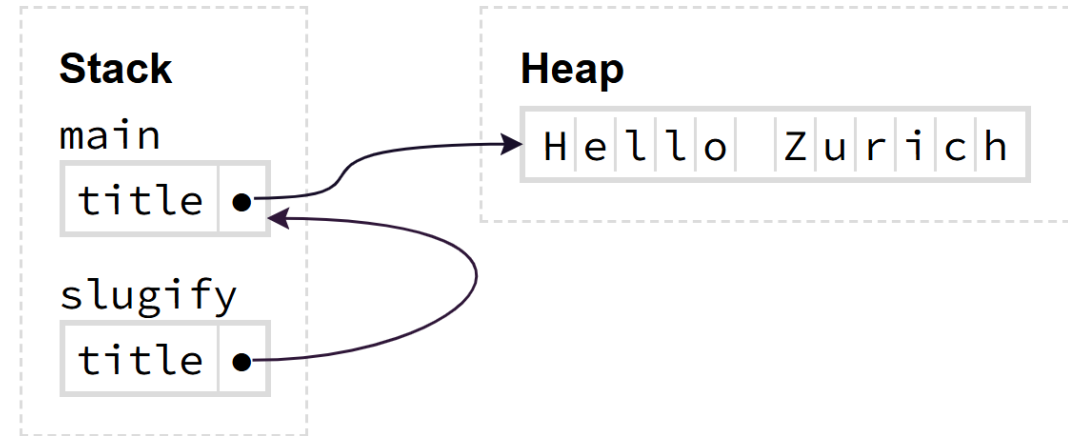
Fixing our example

```
1 fn slugify(title: &String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(&title);  
10    println!("{}", title, slug);  
11 }
```



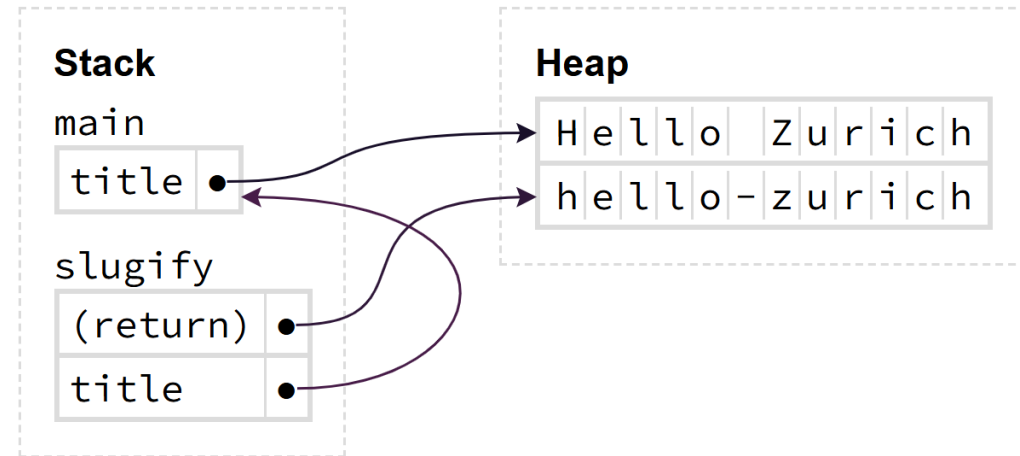
Fixing our example

```
1 fn slugify(title: &String) → String {  
2   title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8   let title = input();  
9   let slug = slugify(&title);  
10  println!("{}", title, slug);  
11 }
```



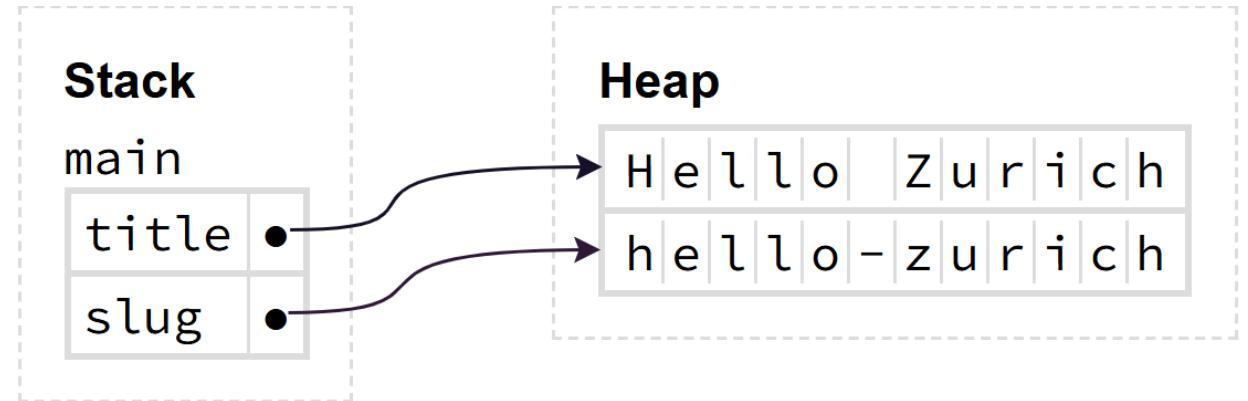
Fixing our example

```
1 fn slugify(title: &String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(&title);  
10    println!("{}", title, slug);  
11 }
```



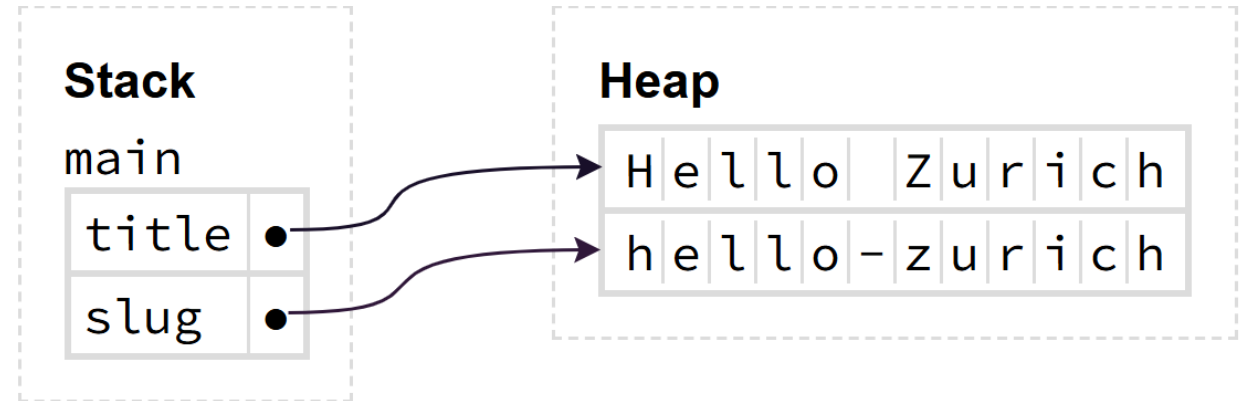
Fixing our example

```
1 fn slugify(title: &String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(&title);  
10    println!("{}", title, slug);  
11 }
```



Fixing our example

```
1 fn slugify(title: &String) → String {  
2     title  
3     .to_lowercase()  
4     .replace(' ', "-")  
5 }  
6  
7 fn main() {  
8     let title = input();  
9     let slug = slugify(&title);  
10    println!("{}", title, slug);  
11 }
```



- We don't give `title` to `slugify`
- We *lend* it
- We get to keep `title` afterwards
- **It compiles**

Mutability and Aliasing



Mutability

	Type	Can reassign (<i>exterior mutability</i>)	Can mutate contents (<i>interior mutability</i>)
let x = ...	T	No	Yes
let mut x = ...	T	Yes	Yes
&x	&T	No	No
& mut x	& mut T	No	Yes

```
1 fn add_exclamation(s: &String) {  
2     s.push_str("!");  
3 }  
4  
5 fn main() {  
6     let s = String::from("hello");  
7     add_exclamation(&s);  
8 }
```

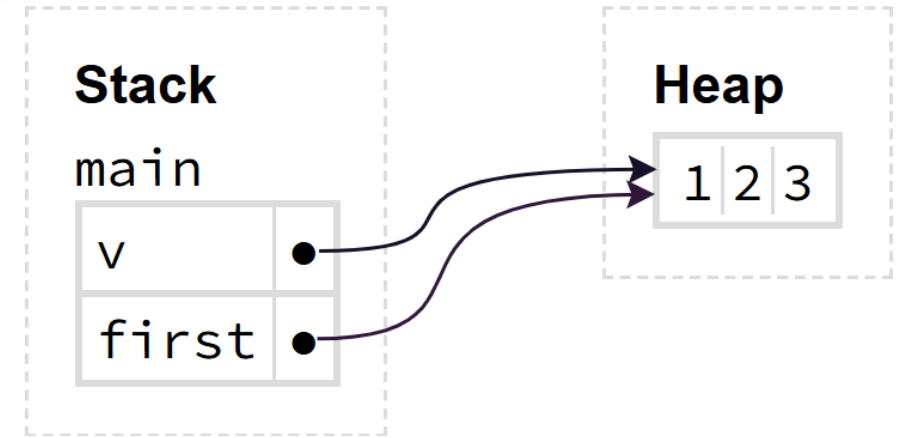
```
1 fn add_exclamation(s: &mut String) {  
2     s.push_str("!");  
3 }  
4  
5 fn main() {  
6     let mut s = String::from("hello");  
7     add_exclamation(&mut s);  
8 }
```

A recipe for disaster

```
1 let mut v = vec![1, 2, 3];  
2 let first = &v[0];  
3 v.push(4);  
4 println!("{}", first);
```

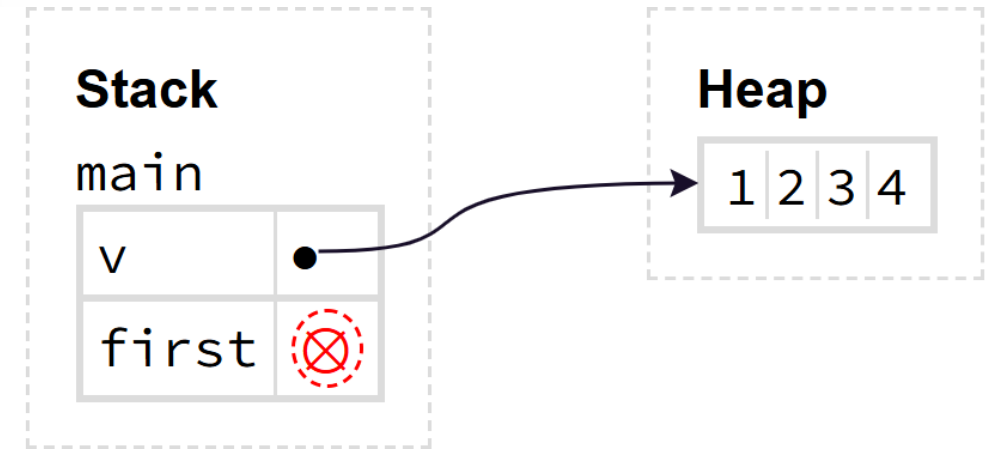
A recipe for disaster

```
1 let mut v = vec![1, 2, 3];  
2 let first = &v[0];  
3 v.push(4);  
4 println!("{}", first);
```



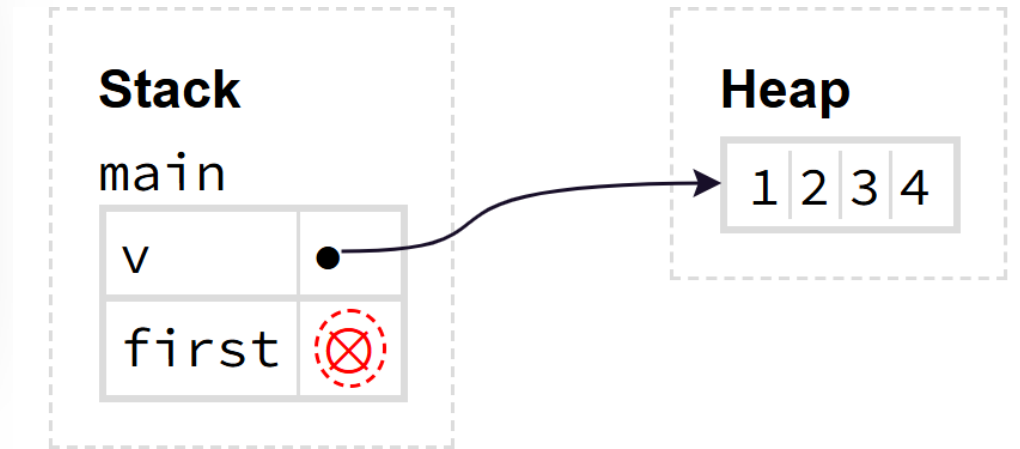
A recipe for disaster

```
1 let mut v = vec![1, 2, 3];  
2 let first = &v[0];  
3 v.push(4);  
4 println!("{}", first);
```



A recipe for disaster

```
1 let mut v = vec![1, 2, 3];  
2 let first = &v[0];  
3 v.push(4);  
4 println!("{}", first);
```

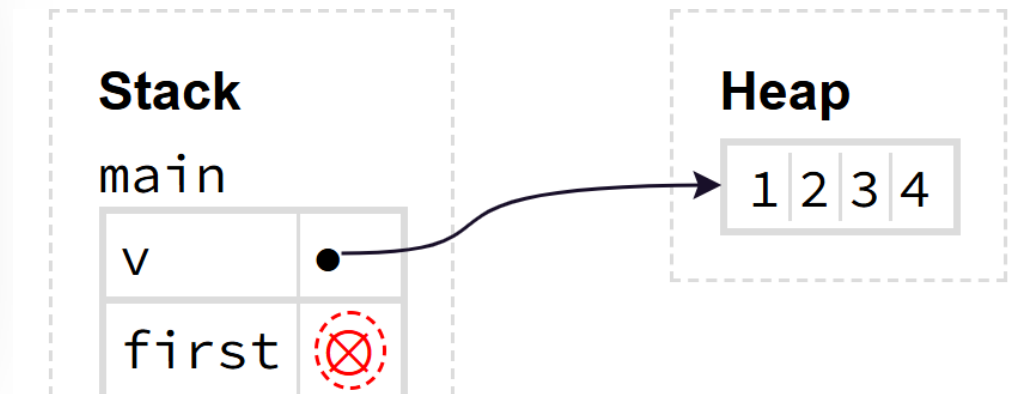


– Pointer safety principle

Data should never be **aliased and mutated** at the same time.

A recipe for disaster

```
1 let mut v = vec![1, 2, 3];
2 let first = &v[0];
3 v.push(4);
4 println!("{}", first);
```



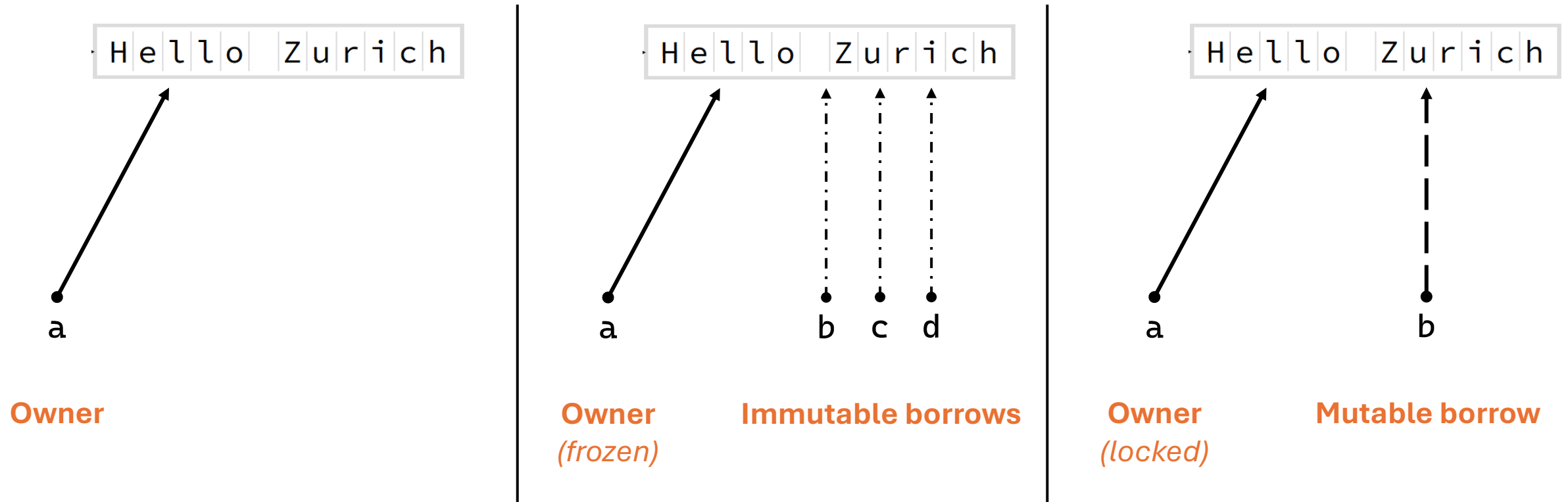
```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
--> src/main.rs:5:5
```

```
4 |     let first = &v[0];
   |               - immutable borrow occurs here
5 |     v.push(4);
   |     ^^^^^^^^^ mutable borrow occurs here
6 |     println!("{}", first);
   |               ----- immutable borrow later used here
```

Pointer safety principle

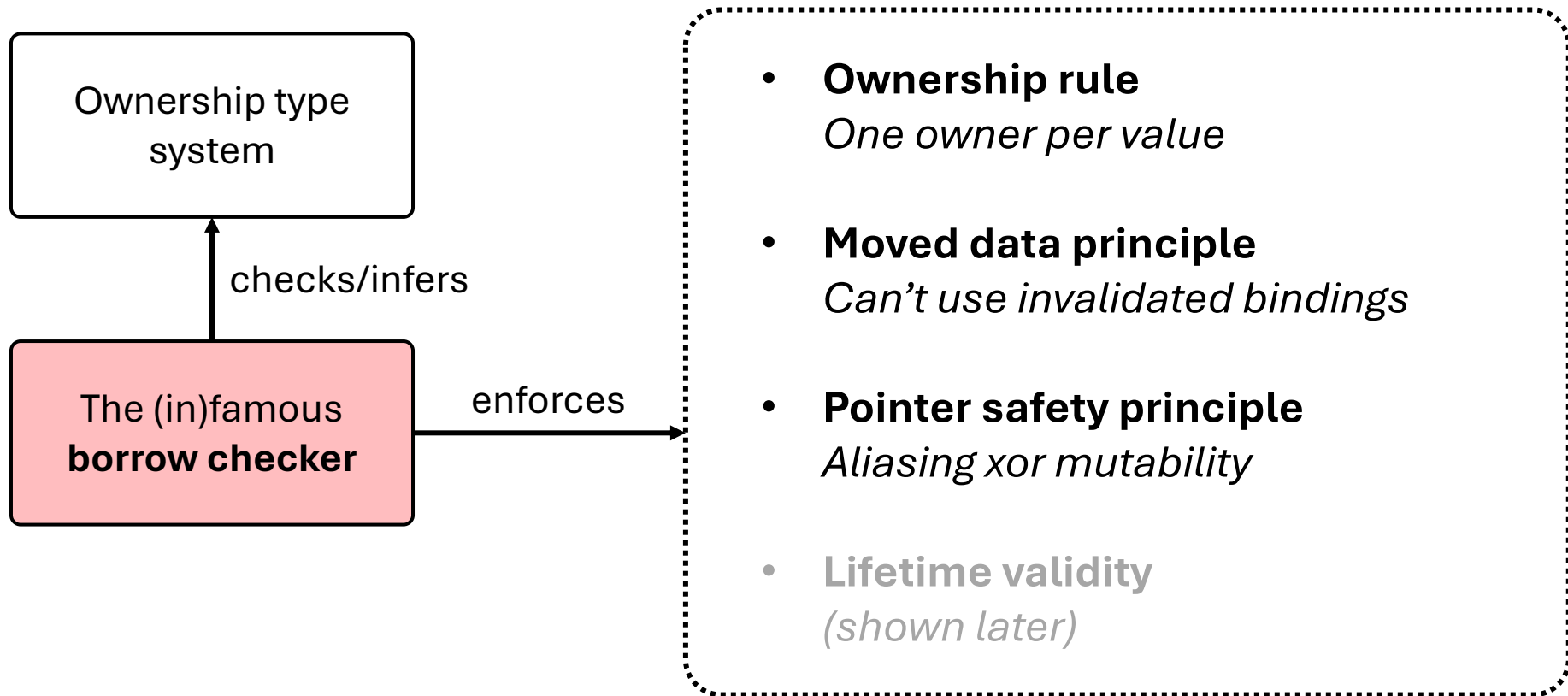
Data should never be **aliased and mutated** at the same time.

“Aliasing xor mutability”

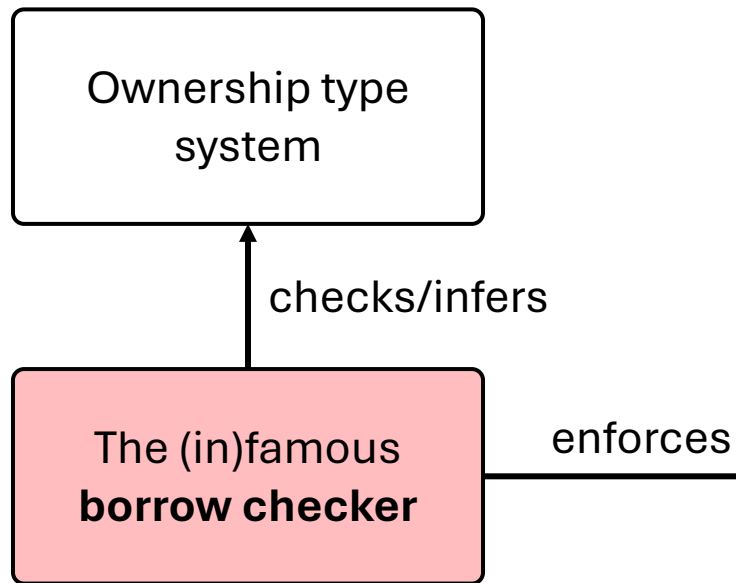


– **Pointer safety principle** –
Data should never be **aliased and mutated** at the same time.

How to enforce all this?



How to enforce all this?



$\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi} p \Rightarrow \{ \omega p' \}$ where $\Delta; \Gamma; \Theta \vdash_{\omega} p \Rightarrow \{ \omega p' \}$ means $\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi} p \Rightarrow \{ \omega p' \}$.

O-SAFEPLACE
 $\frac{\forall \tau' \mapsto \{ \tilde{\tau} \} \in \text{regions}(\Gamma, \Theta), (\forall \omega' p^{\omega}[\pi'] \in \{ \tilde{\tau} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies \pi' \# \pi) \vee (\exists \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma) \wedge \&\&\tau' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma), \pi' \in \{ \pi_e \})))}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi} \pi \Rightarrow \{ \omega \pi \}}$

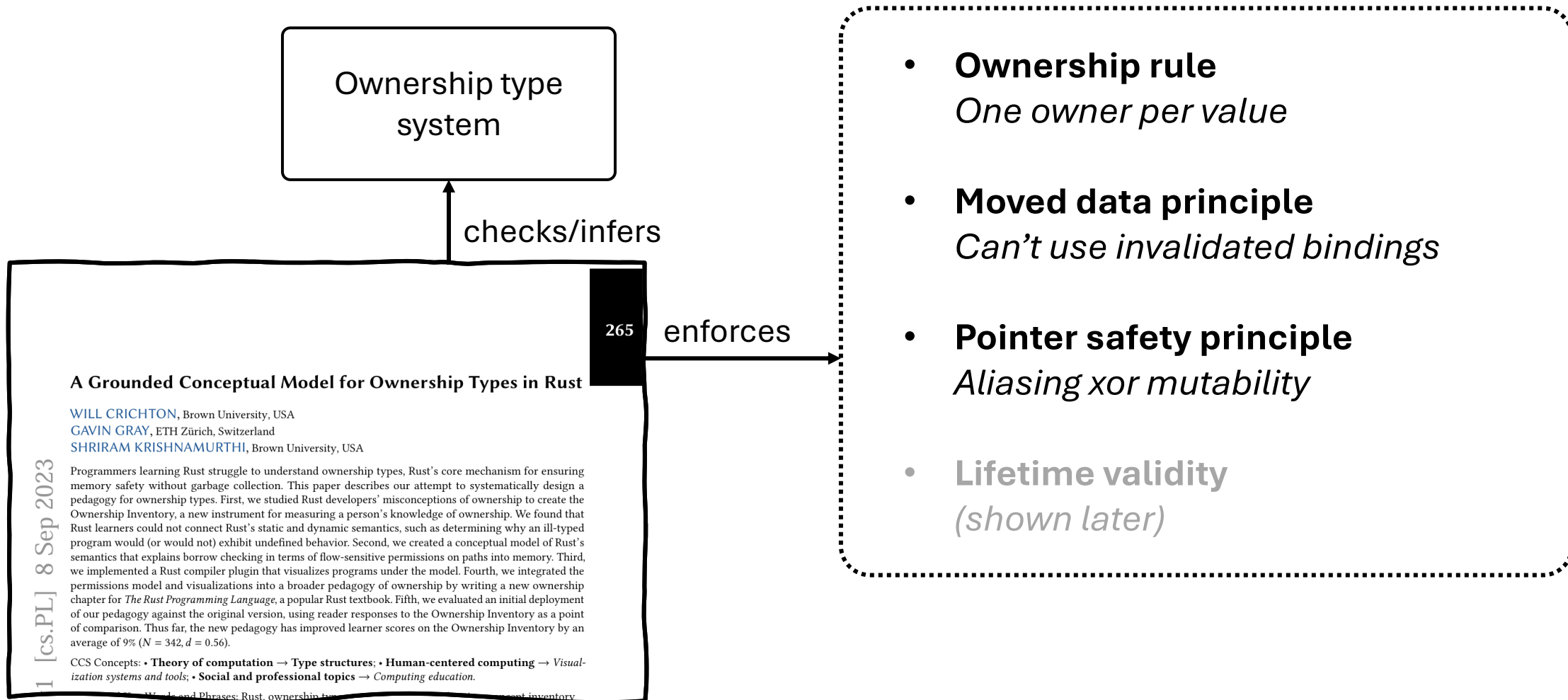
O-DEREF
 $\frac{\Gamma(\pi) = \&\tau \omega_{\pi} \tau_{\pi} \quad \Gamma(\tau) = \{ \omega' p^{\omega'} \} \quad \text{excl} = \{ \pi_j \text{ where } j \in \{ 1, \dots, n \} \mid p_j = p_j^{\omega}[\pi_j] \} \quad \omega \leq \omega_{\pi} \quad \forall i \in \{ 1, \dots, n \}, \Delta; \Gamma; \Theta \vdash_{\omega'}^{\pi_i} \text{excl}, \pi \quad p^{\omega}[\tau_i] \Rightarrow \{ \omega' p_i' \} \quad \forall \tau' \mapsto \{ \tilde{\tau} \} \in \text{regions}(\Gamma, \Theta), (\forall \omega' p^{\omega'} \in \{ \tilde{\tau} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies p' \# p^{\omega}[\pi]) \vee (\exists \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma) \wedge \&\&\tau' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma), \pi' \in \{ \pi_e, \text{excl}, \pi \})))}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi} p^{\omega}[\pi] \Rightarrow \{ \omega p_1', \dots, \omega p_n', \omega p^{\omega}[\pi] \}}$

O-DEREFABS
 $\frac{\Gamma(\pi) = \&\tau \omega_{\pi} \tau_{\pi} \quad \Delta; \Gamma \vdash_{\omega} p^{\omega}[\pi] : \tau \quad \omega \leq \omega_{\pi} \quad \forall \tau' \mapsto \{ \tilde{\tau} \} \in \text{regions}(\Gamma, \Theta), (\forall \omega' p \in \{ \tilde{\tau} \}. (\omega = \text{uniq} \vee \omega' = \text{uniq}) \implies p \# p^{\omega}[\pi]) \vee (\exists \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma) \wedge \&\&\tau' \omega' \tau' \in \Theta \wedge (\forall \pi' : \&\tau' \omega' \tau' \in \text{explode}(\Gamma), \pi' \in \{ \pi_e, \pi \})))}{\Delta; \Gamma; \Theta \vdash_{\omega}^{\pi} p^{\omega}[\pi] \Rightarrow \{ \omega p^{\omega}[\pi] \}}$

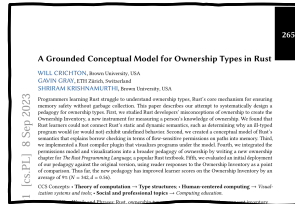


- **Ownership rule**
One owner per value
- **Moved data principle**
Can't use invalidated bindings
- **Pointer safety principle**
Aliasing xor mutability
- **Lifetime validity**
(shown later)

How to enforce all this?



A pedagogy for borrow checking



Crichton, W., Gray, G., Krishnamurthi, S. (2023). A grounded conceptual model for ownership types in Rust. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2), 1224-1252.

- **Ownership inventory:** a concept inventory for measuring understanding of ownership types
- **Conceptual model:** a permission-based model of Rust's static semantics
- **Aquascope:** a tool that visualizes programs under this conceptual model <https://cel.cs.brown.edu/aquascope/>
- **New pedagogy:** a rewritten ownership explanations for *The Rust Programming Language* <https://rust-book.cs.brown.edu/>
- **Empirical evaluation:** an A/B test of the new pedagogy against the original, showing a 9% improvement in scores (N=342, d=0.56)



Cognitive Engineering Lab



Visualizing Program Semantics

Aquascope

Visualizing Rust's ownership types

```
let mut v: Vec<i32> = vec![1, 2, 3];
let num: &i32 = &v[2];
```

Argus

Visualizing Rust's trait system

```
trait Botanical {
    // ...
}
impl<..> Ingredient for B where .. {
    // ...
}
impl<..> Item for T where .. {
    // ...
}
```

Flowistry

Visualizing information flow in Rust

```
pub(crate) fn transfer_function(
    &self,
    state: &mut FlowDomain<'tcx>,
    mutations: Vec<Mutation<'tcx>>,
    location: Location,
) {
    debug!("Applying mutations {mutations:?}");
    let location_domain = self.location_domain();
```

Learning Programming Languages

Rust Book Experiment

Profiling language learning

Quiz ch18-02-refutability / Question 2

Question	Answers
Question 2 Consider the following program: 1. let x: &{i32, i32} = &{10, 1}; Which of the following are refutable patterns for x?	N = 207 • &{ N = 137 • &{

C++ to Rust Phrasebook

Translating across languages

```
#include <concepts>

template <typename T>
concept shape = requires(const T &t) {
    { t.area() } -> std::same_as<double>;
};

template <shape T>
double twiceArea(const T &shape) {
    return shape.area() * 2.0;
}
```

SMoL Tutor

Addressing PL misconceptions

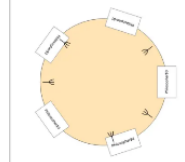
What is the result of running this program?

```
(defvar x 1)
(defvar y (+ x 2))
x
y
```

Explaining Formal Methods

Forge

Alloy, improved!



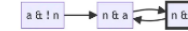
LTL Tutor

Temporal logic, improved!

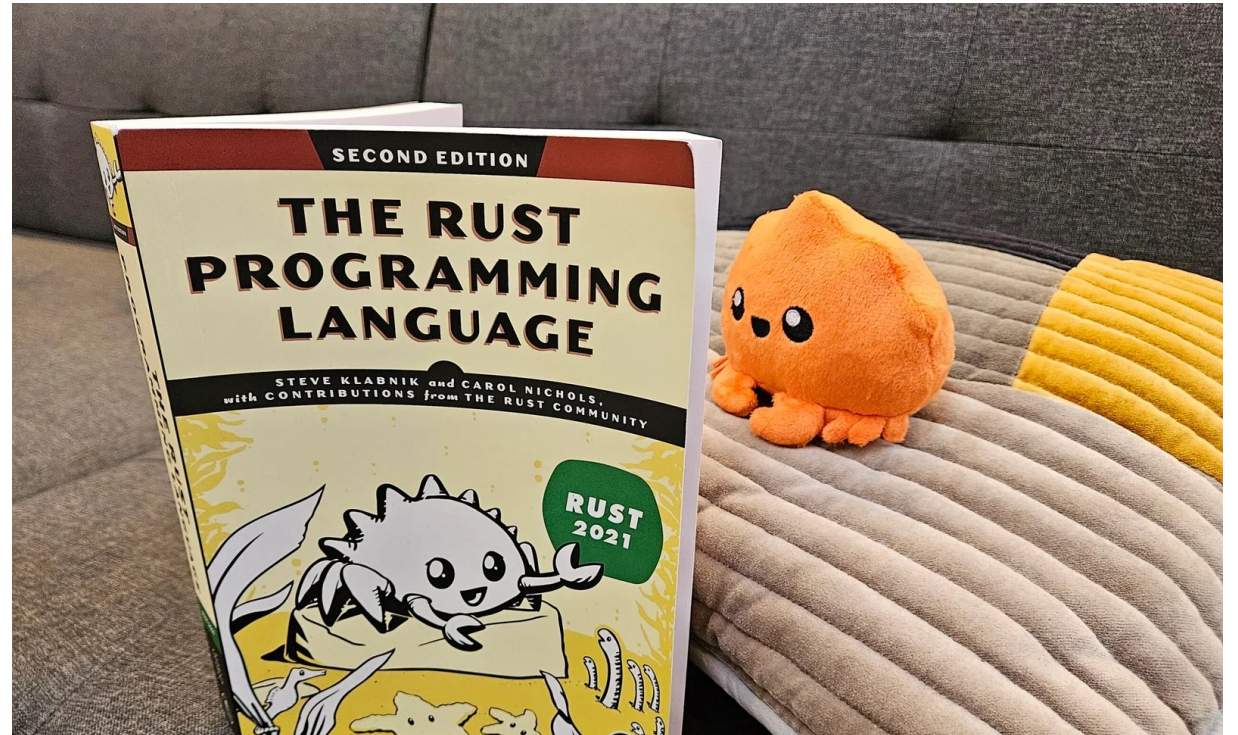
Formula:
(! (F a))



Trace Position



Permission Model of Borrow Checking



The permission system

Each **variable** has up to **3 permissions** at every **program location**

Read: data can be read (and copied)

Write: data can be mutated

Own: data can be moved or dropped

```
fn slugify(title: &String) -> String {  
    title.  
        .to_lowercase()  
        .replace(' ', "-")  
}  
  
fn main() {  
    let title = input();  
    let slug = slugify(&title);  
    println!("{}", title, slug);  
}
```

title ↑ +R - +O
*title ↑ +R - -

title ↓ ✕ - ✕
*title ↓ ✕ - -

title ↑ +R - +O

slug ↑ +R - +O

title ↓ ✕ - ✕
slug ↓ ✕ - ✕

Permission updates

The permission system

place

Each ~~variable~~ has up to **3 permissions** at every **program location**

A **place** is a memory location

- Can appear in LHS of assignment
- Can obtain its address

*(roughly equivalent to
lvalue in C++11)*

```
fn slugify(title: &String) -> String {  
    title.  
        .to_lowercase()  
        .replace(' ', "-")  
}  
  
fn main() {  
    let title = input();  
    let slug = slugify(&title);  
    println!("{}", title, slug);  
}
```

Location	Variable	Permissions
Before slugify call	title	{+R, -, +O}
	*title	{+R, -, -}
After 'let title = input();'	title	{+R, -, +O}
	*title	{+R, -, -}
After 'let slug = slugify(&title);'	title	{+R, -, +O}
	slug	{+R, -, +O}
Before println	title	{+R, -, +O}
	slug	{+R, -, +O}

Permission updates: creating owners

```
1 let mut x = ...;
```



```
1 let      x = ...;
```



Permission updates: dropping owners

```
1 let mut x = ... ;  
2 } // drop(x)
```



```
1 let      x = ... ;  
2 } // drop(x)
```



Permission updates: variable mentions

```
1 x
```

needs **R**

```
1 v.push(4);
```

needs **RW**

Permission updates: moving

```
1 let mut y = x;
```

needs RO

x	➡	R	W	O
y	⬆	+R	+W	+O

(loses all permissions)

(W if mut binding)

(RO always)

Permission updates: creating borrows

```
1 let y = &mut x;
```

&mut x needs **RW**

&x needs **R**

x	➔	R	W	O
y	⬆	+R	-	+O
*y	⬆	+R	+W	-

(**R** if not **mut** borrow)
(**W****O** never)

(**W** if **mut** binding)
(**R****O** always)

(**W** if **mut** borrow)
(**R** always, **O** never)

Example

```
fn slugify(title: &String) -> String {  
    title.  
        .to_lowercase()  
        .replace(' ', "-")  
}  
  
fn main() {  
    let title = input();  
    let slug = slugify(&title);  
    println!("{}", title, slug);  
}
```

The diagram illustrates the state of variables `title` and `slug` across different code blocks, showing how references and mutations are tracked.

- Block 1 (fn slugify):**
 - `title`: Initial state with mutations `+R` (orange), `-` (blue), and `+O` (pink).
 - `*title`: Initial state with mutations `+R` (orange), `-` (blue), and `-` (pink).
- Block 2 (title.to_lowercase().replace):**
 - `title`: Mutated state with `↓` (black), `✗` (orange), `-` (blue), and `∅` (pink).
 - `*title`: Mutated state with `↓` (black), `✗` (orange), `-` (blue), and `-` (pink).
- Block 3 (fn main):**
 - `title`: Initial state with mutations `+R` (orange), `-` (blue), and `+O` (pink).
 - `slug`: Initial state with mutations `+R` (orange), `-` (blue), and `+O` (pink).
- Block 4 (println):**
 - `title`: Mutated state with `↓` (black), `✗` (orange), `-` (blue), and `∅` (pink).
 - `slug`: Mutated state with `↓` (black), `✗` (orange), `-` (blue), and `∅` (pink).

Permission updates: dropping borrows

```
1     let y = &mut x;  
2 } // drop(y)  
3 x
```

x	↺	+R	+W	+O
y	↓	R	-	O
*y	↓	R	W	-

(recovers original)

(invalidated: losses all)

Example

```
fn main() {  
  let mut v = vec![1, 2, 3];  
  
  {  
    let first = &v[0];  
  
    println!("{}", first);  
  
    v.push(4);  
    println!("{:?}", v);  
  }  
}
```

Diagram illustrating memory state and pointer manipulation:

Initial state (after `let mut v = vec![1, 2, 3];`):

v	↑	+R	+W	+O
---	---	----	----	----

State after `let first = &v[0];`:

v	→	R	W	O
first	↑	+R	-	+O
*first	↑	+R	-	-

State after `println!("{}", first);`:

v	↻	R	+W	+O
first	↓	R	-	O
*first	↓	R	-	-

Final state (after `v.push(4);`):

v	↓	R	W	O
---	---	--------------	--------------	--------------

Example

```
fn main() {  
    let mut v = vec![1, 2, 3];  
  
    let first = &v[0];  
  
    v.push(4);  
    println!("{}", first);  
}
```

«— v ↑ +R +W +O

«—

v	→	R	W	O
first	↑	+R	-	+O
*first	↑	+R	-	-



Example

```
fn main() {  
    let mut v =
```

```
    let first = &v[0];
```

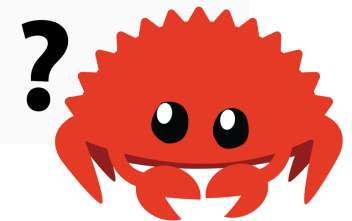
```
    v.push(4);  
    println!("{}", first);  
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable  
--> src/main.rs:5:5
```

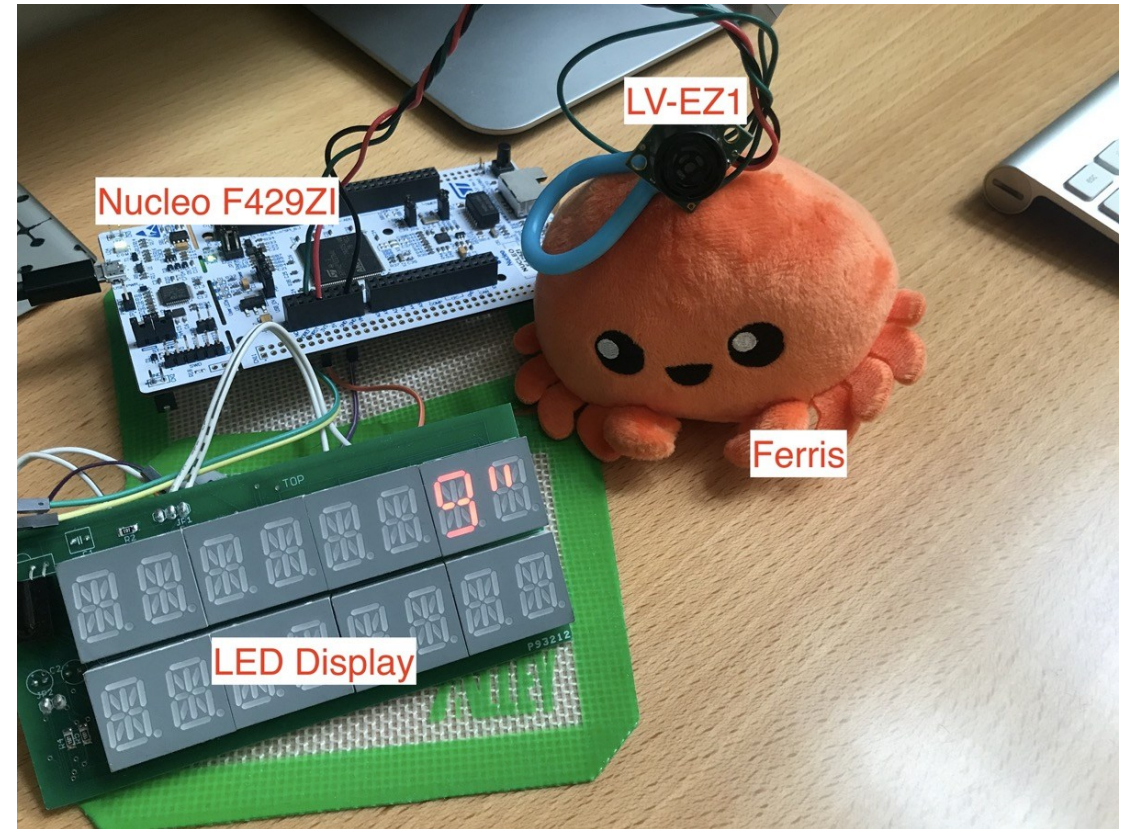
```
4 |     let first = &v[0];  
    |                  - immutable borrow occurs here  
5 |     v.push(4);  
    |     ^^^^^^^^^ mutable borrow occurs here  
6 |     println!("{}", first);  
    |                  ----- immutable borrow later used here
```

«

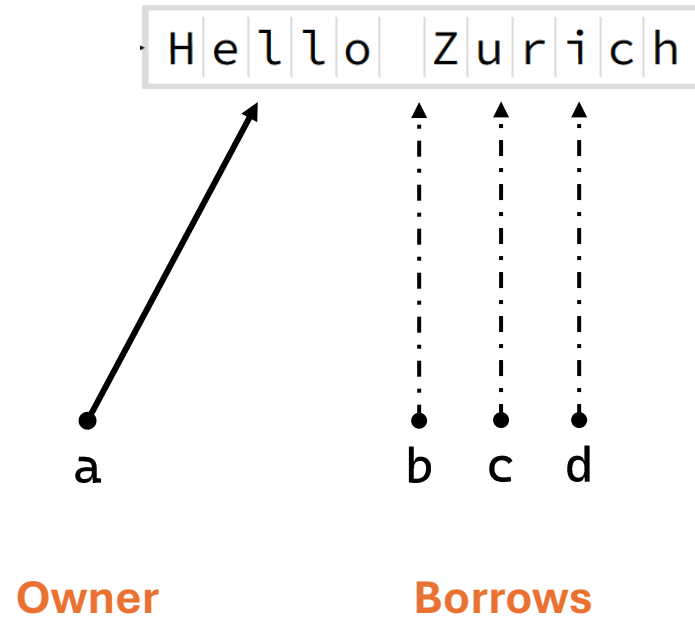
v	→	R	W	Ø
first	↑	+R	-	+O
*first	↑	+R	-	-



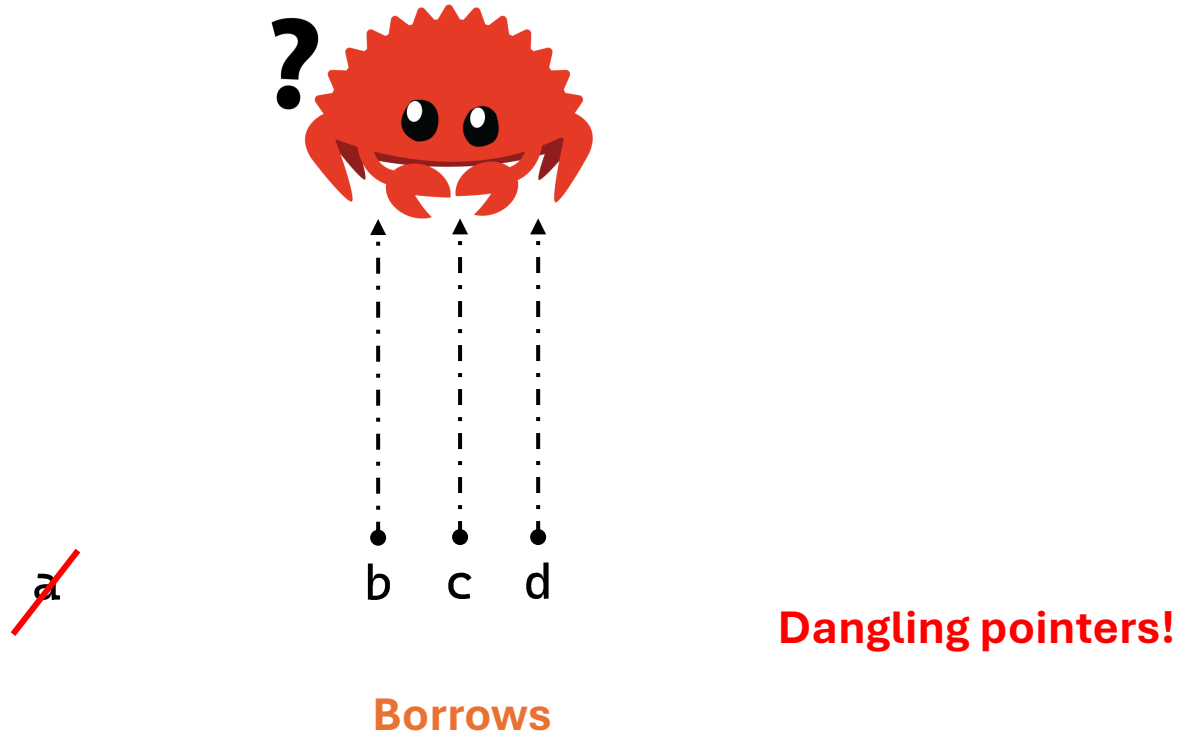
Lifetime Validity



Data must outlive its references



Data must outlive its references



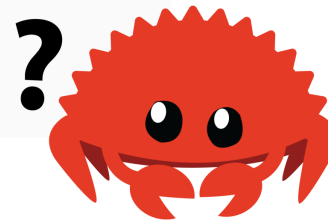
Data must outlive its references

```
fn main() {  
    let s = String::from("Hello");  
  
    let s_ref = &s;  
  
    drop(s);  
    print!("{}", s_ref);  
}
```

« s ↑ +R - +O

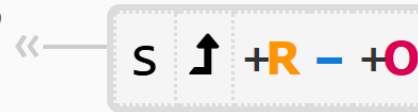
«

s	→	R - Ø
s_ref	↑	+R - +O
*s_ref	↑	+R - -



Data must outlive its references

```
fn main() {  
    let s_ref;  
    {  
        let s = String::from("Hello");  
  
        s_ref = &s;  
    }  
    print!("{}", s_ref);  
}
```



Data must outlive its references

```
fn main() {  
    let s_ref;  
    {  
        let s = String::from("Hello");  
  
        s_ref = &s;  
    }  
    print!("{}", s_ref);  
}
```

```
error[E0597]: `s` does not live long enough  
--> src/main.rs:5:17
```

```
4 |         let s = String::from("Hello");  
    - binding `s` declared here  
5 |         s_ref = &s;  
    - ^^^^^ borrowed value does not live long enough  
6 |     }  
    - `s` dropped here while still borrowed  
7 |     print!("{}", s_ref);  
    - ---- borrow later used here
```

Visualizing lifetimes

```
1 fn main() {  
2     let s_ref;  
3     {  
4         let s = String::from("Hello");  
5         s_ref: & String = & s;  
6     }  
7     print!("{}", s_ref);  
8 }
```

- **Binding is live**
It can be used
- **Lifetime of a binding**
Region where it is live

Visualizing lifetimes

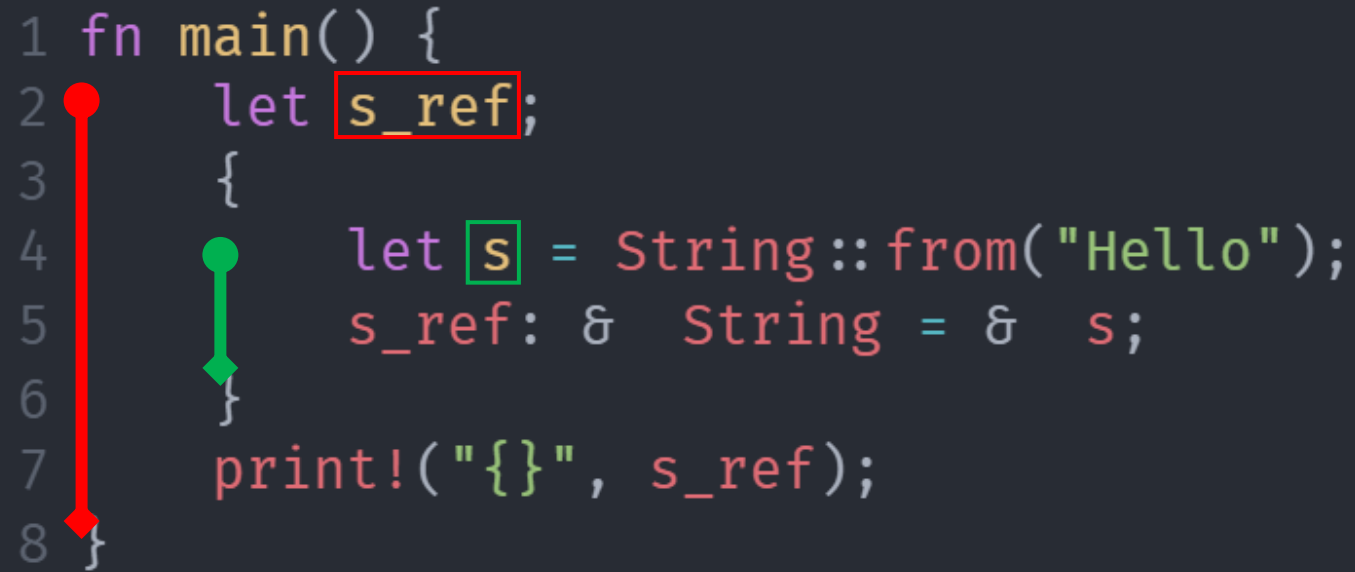
```
1 fn main() {  
2   let s_ref;  
3   {  
4       let s = String::from("Hello");  
5       s_ref: & String = & s;  
6   }  
7   print!("{}", s_ref);  
8 }
```



- **Binding is live**
It can be used
- **Lifetime of a binding**
Region where it is live

Visualizing lifetimes

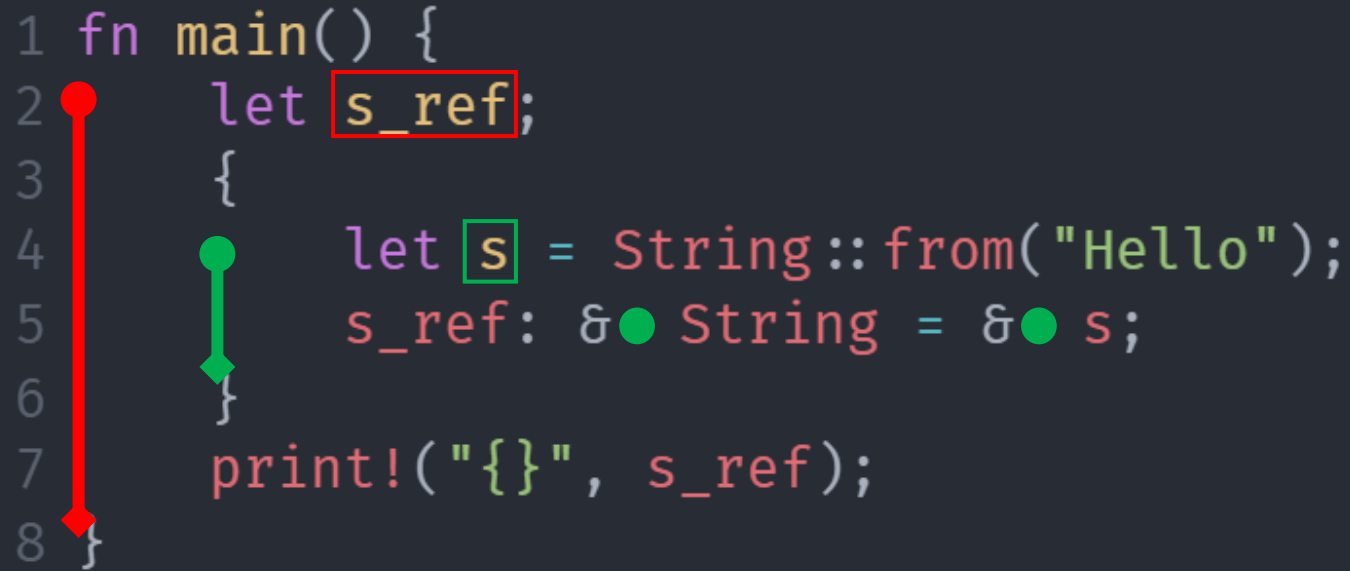
```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: & String = & s;  
6   }  
7   print!("{}", s_ref);  
8 }
```



- **Binding is live**
It can be used
- **Lifetime of a binding**
Region where it is live

Visualizing lifetimes

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = &s;  
6   }  
7   print!("{}", s_ref);  
8 }
```



- **Binding is live**
It can be used
- **Lifetime of a binding**
Region where it is live
- **Every borrow “tracks” a lifetime**
s_ref : &String

Visualizing lifetimes

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = &s;  
6   }  
7   print!("{}", s_ref);  
8 }
```

- **Binding is live**
It can be used
- **Lifetime of a binding**
Region where it is live
- **Every borrow “tracks” a lifetime**
`s_ref : &String`

Complication:
Non-lexical lifetimes (NLL)
since Rust 2018

“Must outlive” constraints

– Lifetime validity principle (I)

Every borrow $\mathbf{r}$: $\&\bullet$ T must **live less than** its referent ($\bullet$), i.e.,

$$\text{lifetime}(*\mathbf{r}) = \bullet \supseteq_{\text{“outlives”}} \text{lifetime}(\mathbf{r})$$

“Must outlive” constraints

– Lifetime validity principle (I)

Every borrow $r: \&\bullet T$ must **live less than** its referent ($\bullet$), i.e.,

$$\text{lifetime}(*r) = \bullet \supseteq_{\text{“outlives”}} \text{lifetime}(r)$$

```
s_ref: &● String = &● s;
```

“Must outlive” constraints

– Lifetime validity principle (I)

Every borrow $r: \&\bullet T$ must **live less than** its referent ($\bullet$), i.e.,

$$\text{lifetime}(*r) = \bullet \supseteq_{\text{“outlives”}} \text{lifetime}(r)$$

$$\bullet \supseteq \text{lifetime}(s_ref)$$

```
s_ref: &● String = &● s;
```

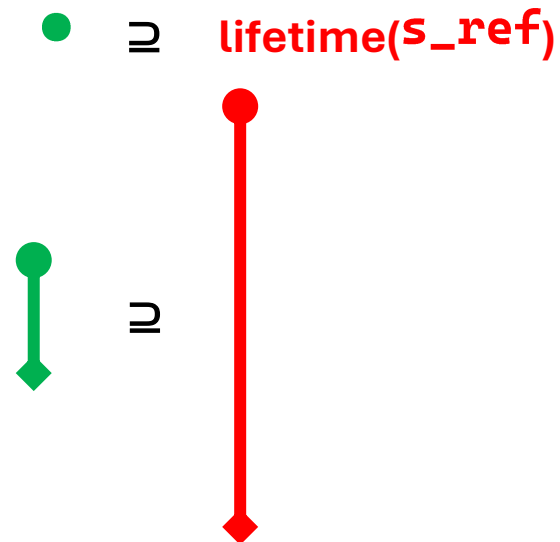
“Must outlive” constraints

– Lifetime validity principle (I)

Every borrow $r: \&\bullet T$ must **live less than** its referent ($\bullet$), i.e.,

$$\text{lifetime}(*r) = \bullet \supseteq_{\text{“outlives”}} \text{lifetime}(r)$$

```
s_ref: &● String = &● s;
```



“Must outlive” constraints

– Lifetime validity principle (I)

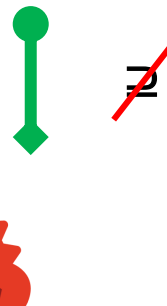
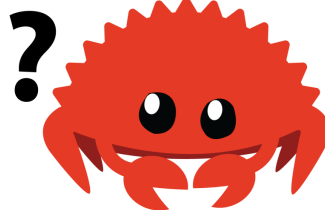
Every borrow $r: \&\bullet T$ must **live less than** its referent ($\bullet$), i.e.,

$$\text{lifetime}(*r) = \bullet \supseteq \text{lifetime}(r)$$

“outlives”

$$\bullet \supseteq \text{lifetime}(s_ref)$$

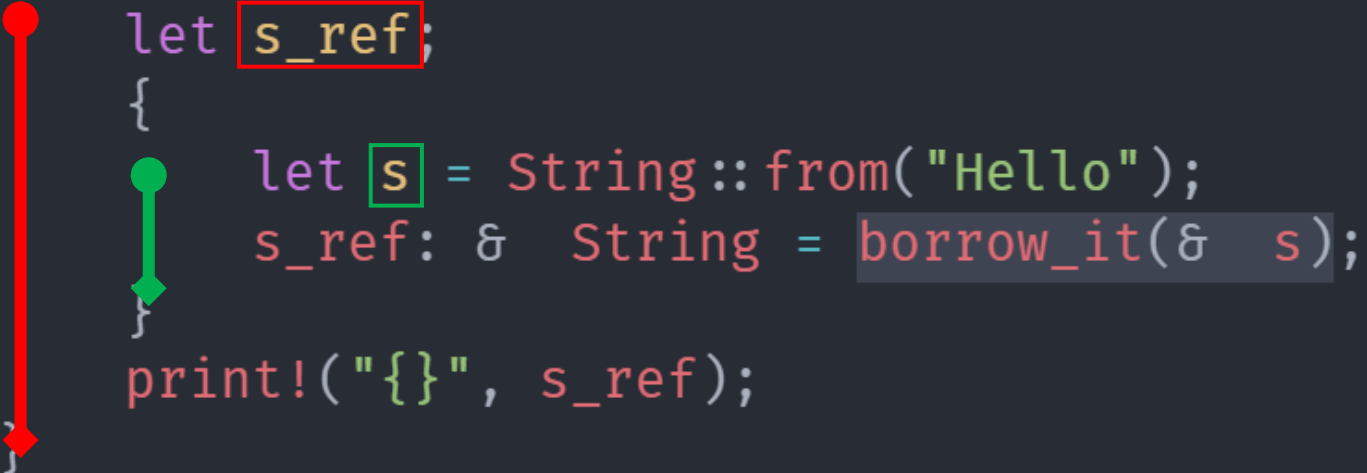
```
s_ref: &● String = &● s;
```



```
error[E0597]: `s` does not live long enough
--> src/main.rs:5:17
4 |         let s = String::from("Hello");
  |         - binding `s` declared here
5 |         s_ref = &s;
  |               ^^ borrowed value does not live long enough
6 |     }
  |     - `s` dropped here while still borrowed
7 |     print!("{}", s_ref);
  |               ----- borrow later used here
```

Lifetimes across function boundaries

```
1 fn main() {  
2     let s_ref;  
3     {  
4         let s = String::from("Hello");  
5         s_ref: & String = borrow_it(& s);  
6     }  
7     print!("{}", s_ref);  
8 }
```



Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```



```
1 fn borrow_it(s: &String) -> &String
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

```
1 fn borrow_it(s: &String) -> &String
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```



```
1 fn borrow_it(s: &String) -> &String
```



Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

s_ref: $\&\text{String}$
 $\bullet \supseteq \bullet$

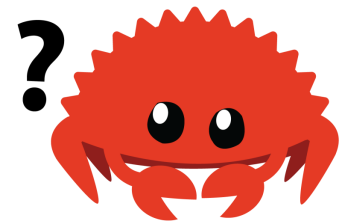
```
1 fn borrow_it(s: &String) → &String
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

s_ref: &String

• ≠ •



```
1 fn borrow_it(s: &String) -> &String
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

```
1 fn borrow_it(s: &String) -> &String
```

Lifetimes across function boundaries



```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

```
1 fn borrow_it(s: &String) → &String
```

Lifetimes across function boundaries



```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

```
1 fn borrow_it(s: &String) → &String
```

Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

s_ref: $\&$ String
 $\bullet \supseteq \bullet$

```
1 fn borrow_it(s: &String) → &String
```

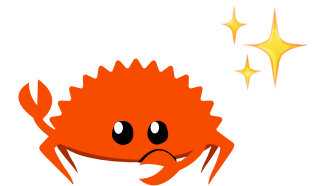
Lifetimes across function boundaries

```
1 fn main() {  
2   let s_ref;  
3   {  
4     let s = String::from("Hello");  
5     s_ref: &String = borrow_it(&s);  
6   }  
7   print!("{}", s_ref);  
8 }
```

```
1 fn borrow_it(s: &String) -> &String
```

s_ref: &String

• $\supseteq$ •



Lifetime annotations

```
1 fn longest(lhs: &String, rhs: &String) → &String {  
2     if lhs.len() > rhs.len() {  
3         lhs  
4     } else {  
5         rhs  
6     }  
7 }
```

Lifetime annotations

```
1 fn longest(lhs: &String, rhs: &String) → &String {  
2     if lhs.len() > rhs.len() {  
3         lhs  
4     } else {  
5         rhs  
6     }  
7 }
```

error[E0106]: missing lifetime specifier

--> src/main.rs:1:43

```
1 | fn longest(lhs: &String, rhs: &String) -> &String {  
    |          -----          -----      ^ expected named lifetime parameter
```

= help: this function's return type contains a borrowed value, but the signature does not say what its lifetime is relative to
help: consider introducing a named lifetime parameter

```
1 | fn longest<'a>(lhs: &'a String, rhs: &'a String) -> &'a String {  
    |          ++++      ++              ++              ++
```

Lifetime annotations

```
1 fn longest<'a>(lhs: &'a String, rhs: &'a String) → &'a String {  
2     if lhs.len() > rhs.len() {  
3         lhs  
4     } else {  
5         rhs  
6     }  
7 }
```

```
1 | fn longest(lhs: &String, rhs: &String) -> &String {  
  | -----  
  |                                     ^ expected named lifetime parameter  
  |  
  | = help: this function's return type contains a borrowed value, but the signature does not say what  
  | help: consider introducing a named lifetime parameter  
1 | fn longest<'a>(lhs: &'a String, rhs: &'a String) -> &'a String {  
  |          ++++      ++              ++              ++
```

Bringing lifetimes to the permission model

Read: data can be read (and copied)

Write: data can be mutated

Own: data can be moved or dropped

→ **Flow:** reference can be safely dereferenced

```
1 r: &●T
```

has **F** until ● ends

until its referent is dropped

Needs **F**:

- Reading/writing through a reference
- Returning a reference
- Passing a reference to a function

(**r** has **F** when ● $\geq$ **lifetime(r)**)

Bringing lifetimes to the permission model

Read: data can be read (and copied)

Write: data can be mutated

Own: data can be moved or dropped

→ **Flow:** reference can be safely dereferenced

There are lifetime errors not caught by this permission model.

```
1 r: &●T
```

has **F** until ● ends

until its referent is dropped

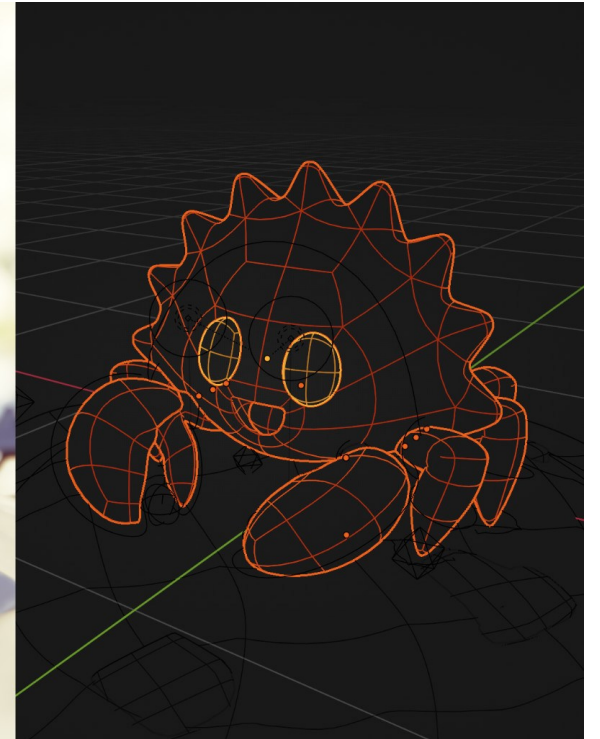
Needs **F**:

- Reading/writing through a reference
- Returning a reference
- Passing a reference to a function

(r has **F** when ● $\supseteq$ **lifetime(r)**)

Formal Foundations

(at a very high level)



Affine type systems

```
1 let x = ... ;  
2 let y = x ;  
3 consume(y) ;
```

- ✓ normal
- ✓ affine
- ✓ linear

```
1 let x = ... ;  
2 let y = x ;
```

- ✓ normal
- ✓ affine
- ✗ linear

```
1 let x = ... ;  
2 let y = x ;  
3 let z = x ;
```

- ✓ normal
- ✗ affine
- ✗ linear

Type systems	Exchange	Weakening	Contraction	Use
Ordered	—	—	—	Exactly once, in the order it was introduced
Linear	Allowed	—	—	Exactly once
Affine	Allowed	Allowed	—	At most once ^[a]
Relevant	Allowed	—	Allowed	At least once
Normal	Allowed	Allowed	Allowed	Arbitrarily ^[b]

[Substructural type system - Wikipedia](#)

Affine type systems

```
1 let x = ... ;  
2 let y = x ;  
3 consume(y) ;
```

- ✓ normal
- ✓ affine
- ✓ linear

```
1 let x = ... ;  
2 let y = x ;
```

- ✓ normal
- ✓ affine
- ✗ linear

```
1 let x = ... ;  
2 let y = x ;  
3 let z = x ;
```

- ✓ normal
- ✗ affine
- ✗ linear

Type systems	Exchange	Weakening	Contraction	Use
Ordered	—	—	—	Exactly once, in the order it was introduced
Linear	Allowed	—	—	Exactly once
Affine	Allowed	Allowed	—	At most once ^[a]
Relevant	Allowed	—	Allowed	At least once
Normal	Allowed	Allowed	Allowed	Arbitrarily ^[b]

[Substructural type system - Wikipedia](#)



Moved data principle

A variable **cannot be used** after it has been **moved** (invalidated).

Affine type systems

```
1 let x = ... ;  
2 let y = x ;  
3 consume(y);
```

- ✓ normal
- ✓ affine
- ✓ linear

```
1 let x = ... ;  
2 let y = x ;
```

- ✓ normal
- ✓ affine
- ✗ linear

```
1 let x = ... ;  
2 let y = x ;  
3 let z = x ;
```

- ✓ normal
- ✗ affine
- ✗ linear

Type systems	Exchange	Weakening	Contraction	Use
Ordered	—	—	—	Exactly once, in the order it was introduced
Linear	Allowed	—	—	Exactly once
Affine	Allowed	Allowed	—	At most once ^[a]
Relevant	Allowed	—	Allowed	At least once
Normal	Allowed	Allowed	Allowed	Arbitrarily ^[b]

[Substructural type system - Wikipedia](#)



Moved data principle
A variable **cannot be used** after it has been **moved** (invalidated).

for some types

Escaping affine constraints: `Copy` and `Clone`

	<code>T</code>	<code>&'a T</code>	<code>&'a mut T</code>
<code>Copy</code> (implicit)	Opt-in	Yes	No
<code>Clone</code> (explicit)	Opt-in	Yes	No
Weakening (discard)	Yes	Yes	Yes
Contraction (duplicate)	Opt-in	Yes	No
Consumption (invalidated on use)	Opt-in	No	Yes

Borrow / Share

```
1 {  
2   let a = String::from("Hello");  
3   let b: &String = &a;  
4 }
```

Move

```
1 {  
2   let a = String::from("Hello");  
3   let b: String = a;  
4 }
```

(Explicit) Copy

```
1 {  
2   let a = String::from("Hello");  
3   let b: String = a.clone();  
4 }
```

Translation to MIR

```
1 fn slugify(title: &String) → String {
2     title
3     .to_lowercase()
4     .replace(' ', "-")
5 }
6
7 fn main() {
8     let title = input();
9     let slug = slugify(&title);
10    println!("{}", title, slug);
11 }
```

```
1 fn slugify(_1: &str) → String {
2     let _0: String;
3     let _2: &str;
4     let _3: &String;
5     let _4: String;
6
7     bb0: _4 = str::to_lowercase(copy _1) → bb1;
8     bb1: _3 = &_4 → bb2;
9     bb2: _2 = String::deref(move _3) → bb3;
10    bb3: _0 = str::replace(copy _2, ' ', "-") → bb4;
11    bb4: drop(_4) → bb5;
12    bb5: return;
13 }
14
15 fn main() {
16     let _1: String;
17     let _2: String;
18     let _3: &str;
19     let _4: &String;
20
21    bb0: _1 = input() → bb1;
22    bb1: _4 = &_1 → bb2;
23    bb2: _3 = String::deref(copy _4) → bb3;
24    bb3: _2 = slugify(copy _3) → bb4;
25    // call to print!
26    bb4: drop(_2) → bb5;
27    bb5: drop(_1) → bb6;
28    bb6: return;
29 }
```

Handling lifetimes: Polonius

```
loan_issued_at(L, P).      % &x at point P creates loan L
loan_invalidated_at(L, P). % mutation/drop at P invalidates L
cfg_edge(P, Q).           % control flow edge P → Q
live(L, Q).               % L is needed at Q (liveness oracle)
```

```
requires(L, P) :- loan_issued_at(L, P).
```

```
requires(L, Q) :- requires(L, P),
                  cfg_edge(P, Q),
                  live(L, Q).
```

```
error(L, P) :- requires(L, P),
               loan_invalidated_at(L, P).
```

*I am but a human.
The borrow checker is not.*



Papers We LoveSM $f(x)=x$

Daniel Galán Pascual 